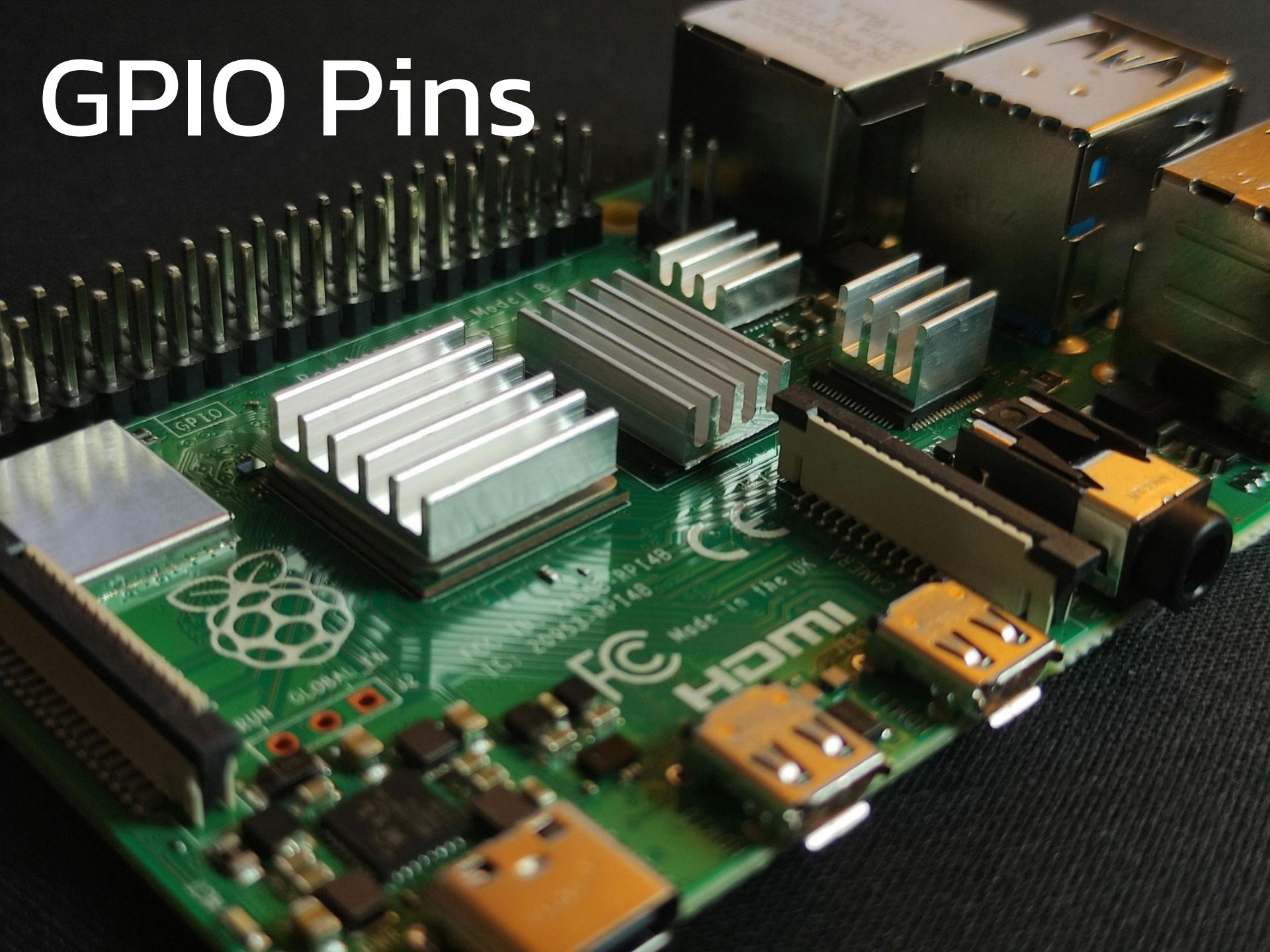
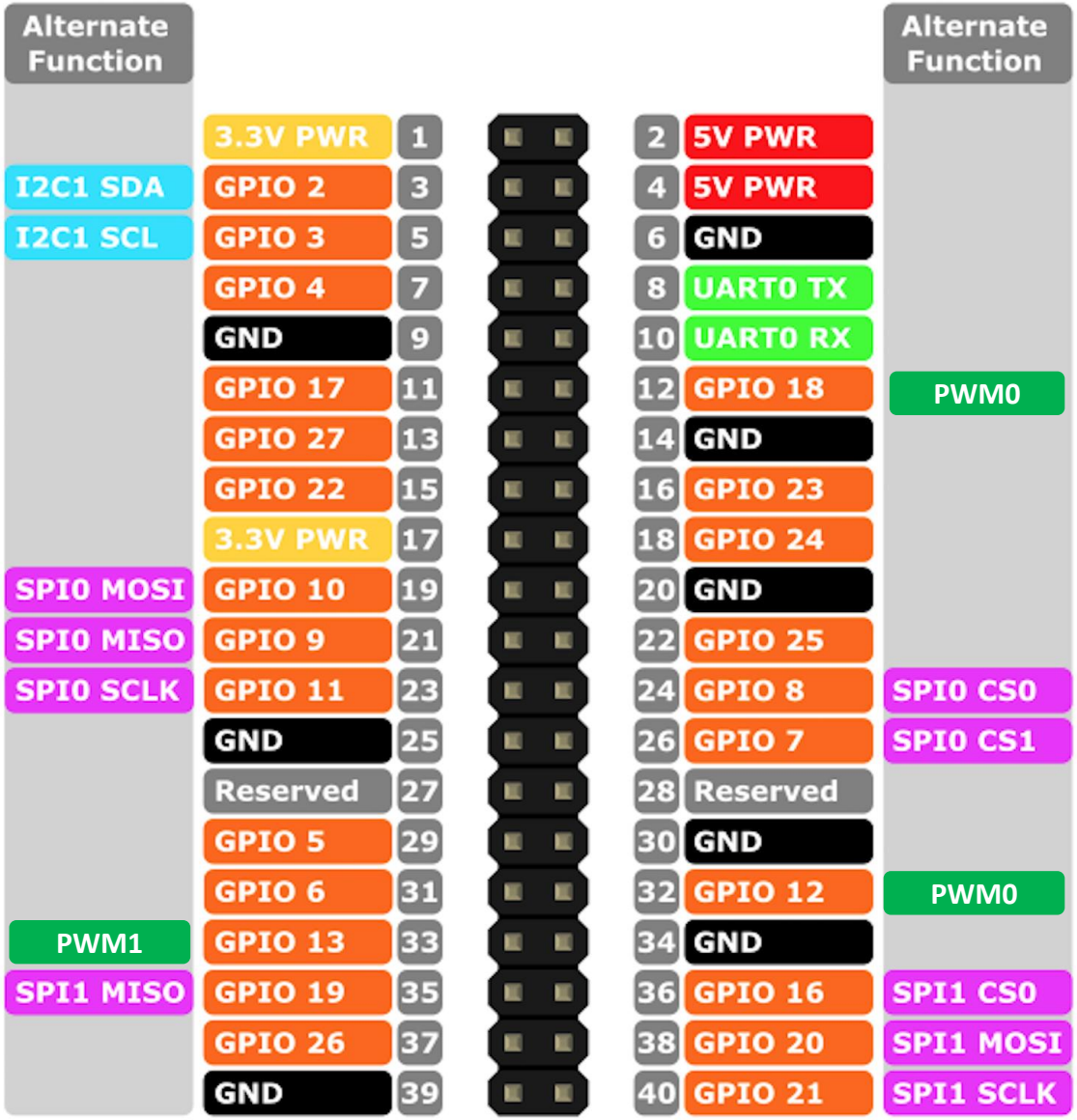
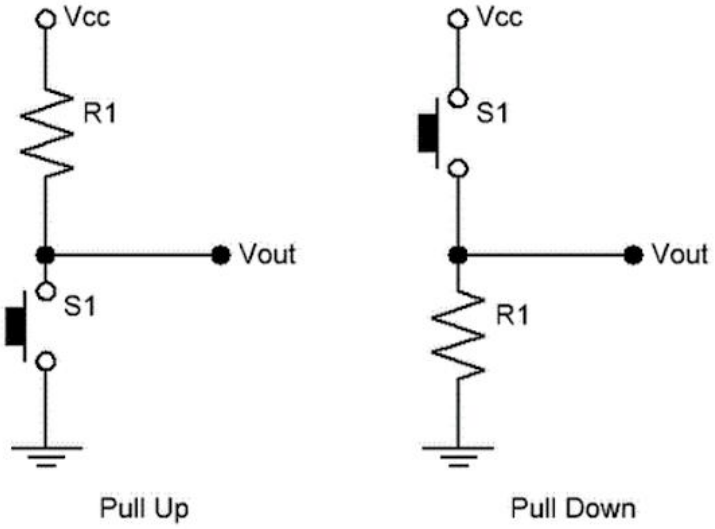
# GPIO Pins



# 1. GPIO Pin ( GPIO : General Purpose Input / Output )



# 2. GPIO : Digital Input




**Pull Up**

Vcc = +3.3V

กด switch Vout = **LOW**

ปล่อย switch Vout = **HIGH**

**Pull Down**

Vcc = +3.3V

กด switch Vout = **HIGH**

ปล่อย switch Vout = **LOW**

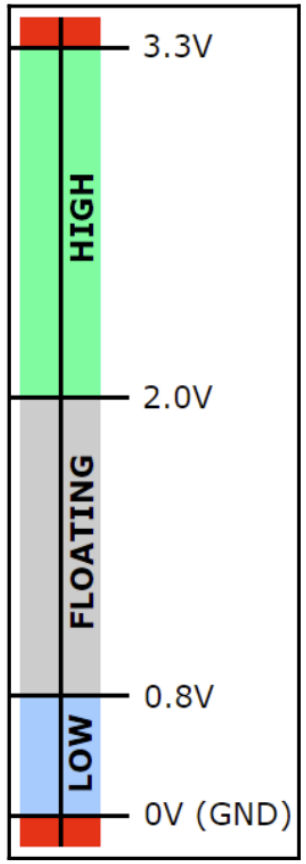


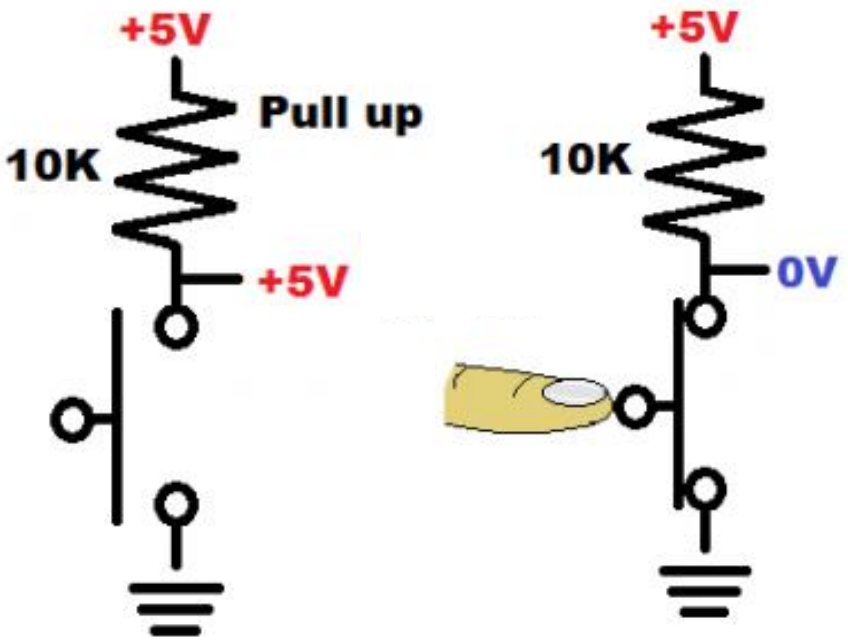

Figure 6.4 – Digital input trigger voltages

**ข้อควรระวัง** เนื่องจาก Raspi ทำงานที่ 3.3V

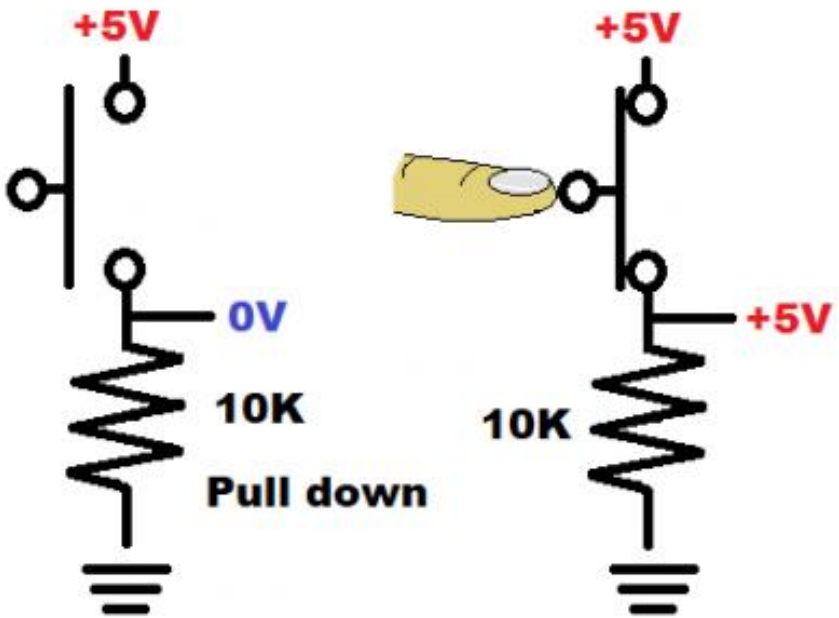
ดังนั้น สัญญาณที่ป้อนเข้า ต้องไม่เกิน 3.3 V

# Pull up & Pull down

## Pull up




## Pull down

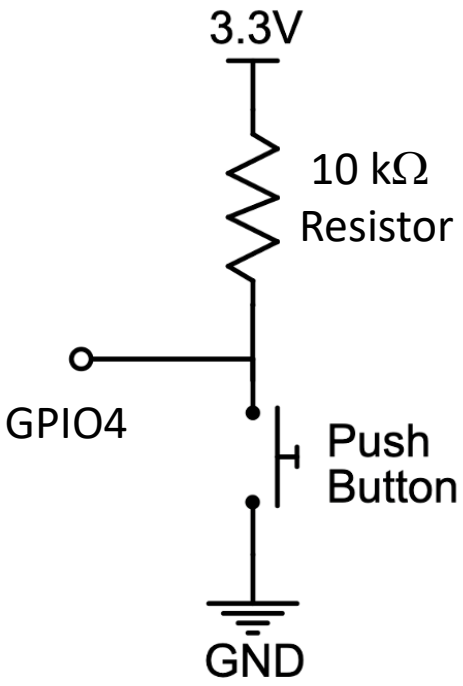


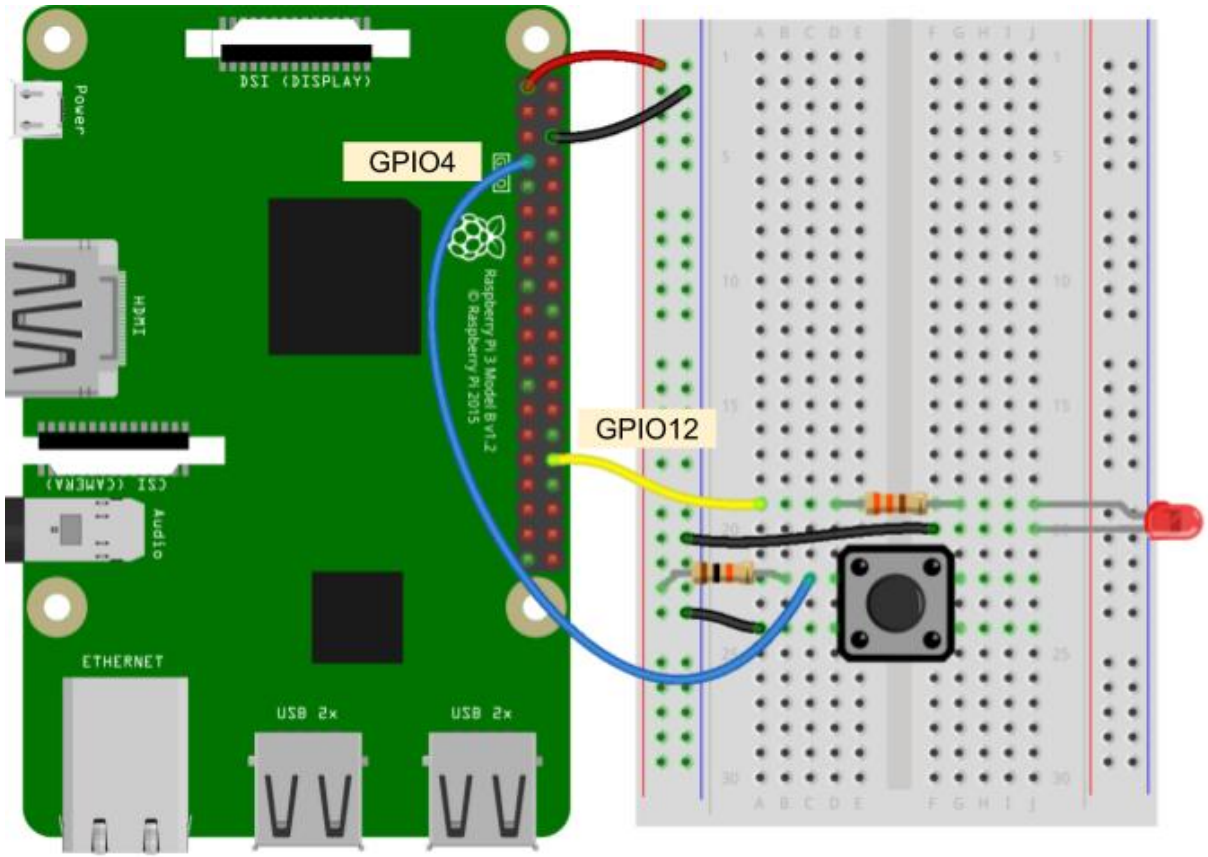

**ข้อควรระวัง** เนื่องจาก Raspi ทำงานที่ 3.3V

ดังนั้น สัญญาณที่ป้อนเข้า ต้องไม่เกิน 3.3 V

# การทดลองที่ 1. Digital Input

- ต่อตัวต้านทาน 10 กิโลโอห์ม และ Switch และต่อเข้าขา GPIO4 และต่อตัวต้านทาน 330 โอห์ม กับ LED เข้าขา GPIO12 ดังรูป



# การทดลองที่ 1. Digital Input

การกำหนดค่าให้ GPIO 4 เป็น Input ใช้คำสั่ง

```
GPIO.setup(4,GPIO.IN)        # กำหนด GPIO 4 เป็นขา Input
```

และการอ่านค่าจาก GPIO 4 ใช้คำสั่ง

```
val = GPIO.input(4)          # ค่าที่อ่านได้ มาเก็บในตัวแปร val โดย
                             จะได้ค่าเป็น False หรือ True
```

**แบบฝึกหัด**

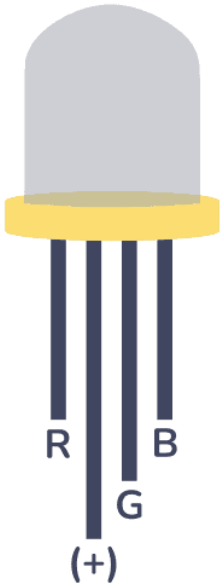
เมื่อกด switch ให้ LED ติด และเมื่อปล่อย switch ให้ LED ดับ

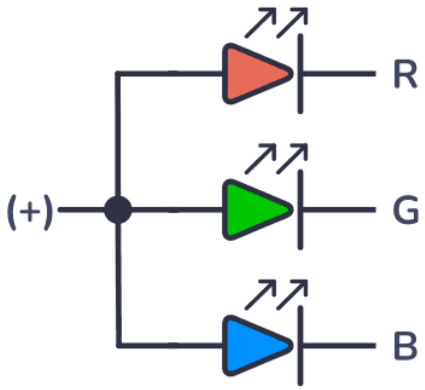
# การทดลองที่ 2. Digital Input

ต่อ Switch จำนวน 3 ตัว

เมื่อกด switch แต่ละตัว ให้ Color LED ติดคนละสี

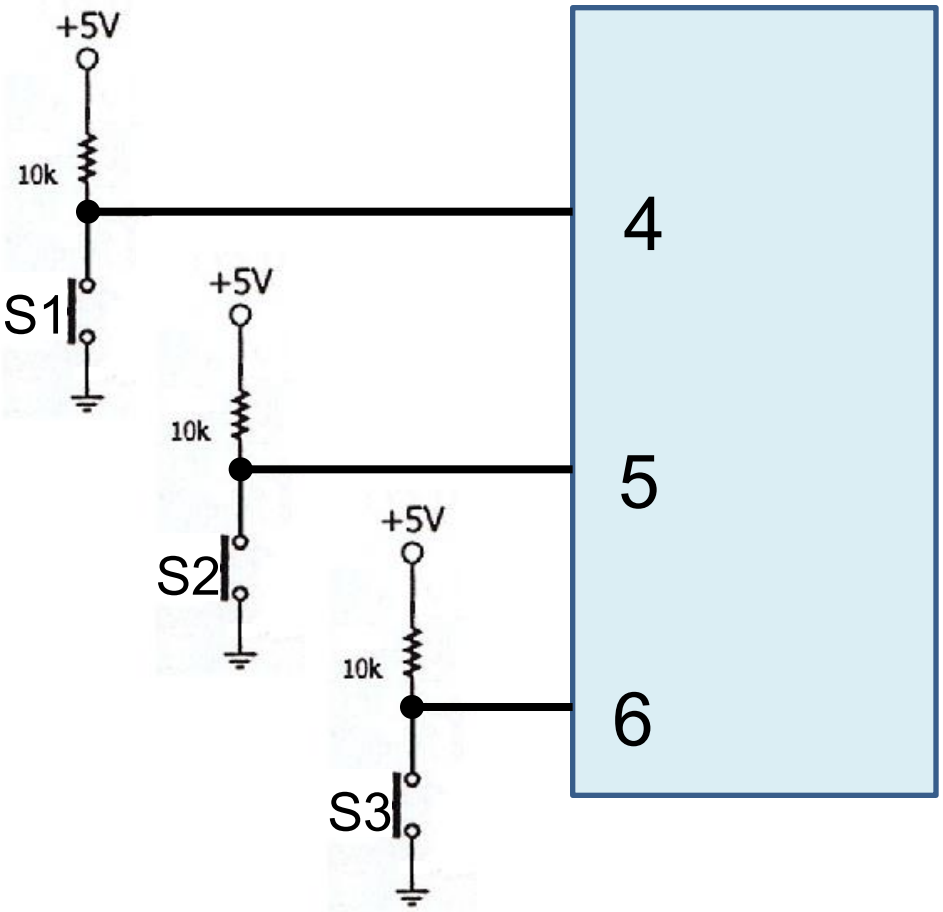
เช่น Red, Green , Blue และเมื่อปล่อย switch ให้ Color LED ดับ





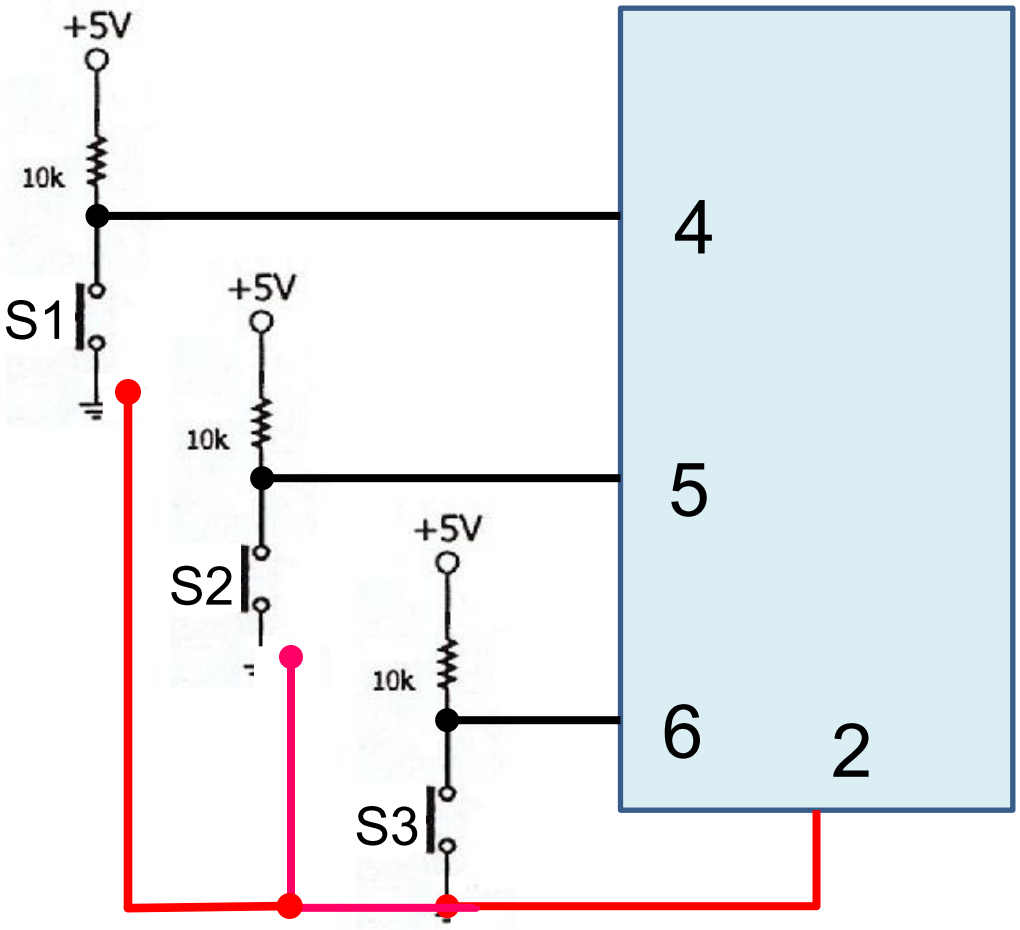

# 3. Multiple Switch => Matrix Switch




```
GPIO.setup(4,GPIO.IN)
GPIO.setup(5,GPIO.IN)
GPIO.setup(6,GPIO.IN)
```

```
if  GPIO.input(4) == False:
  print("Press S1")
elif  GPIO.input(5) == False:
  print("Press S2")
elif  GPIO.input(6) == False:
  print("Press S3")
else:
  print("None")
```

# Multiple Switch => Matrix Switch



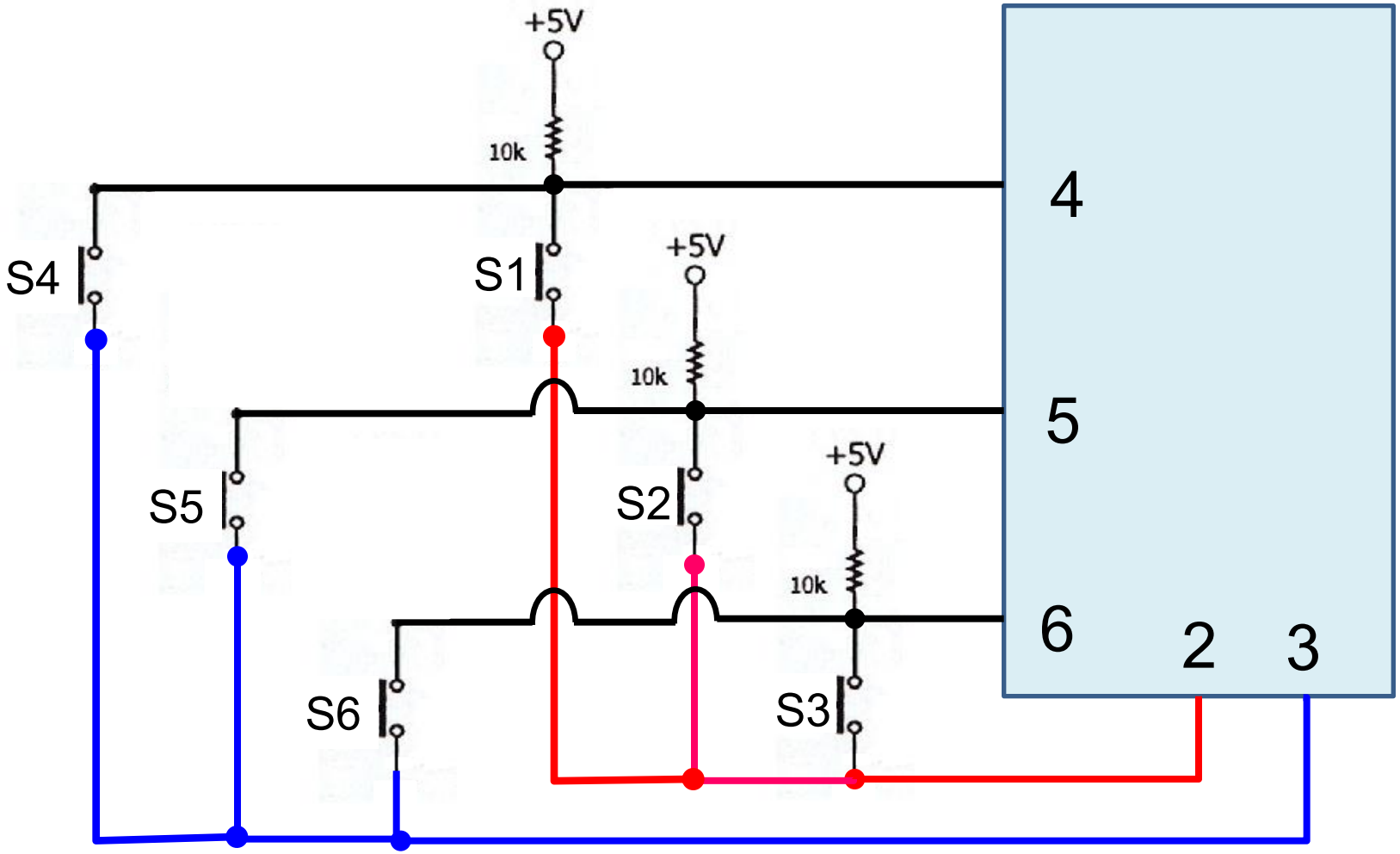

```
GPIO.setup(4,GPIO.IN)
GPIO.setup(5,GPIO.IN)
GPIO.setup(6,GPIO.IN)

GPIO.setup(2,GPIO.OUT)

GPIO.output(2,False)
    if  GPIO.input(4) == False:
      print("Press S1")
    elif  GPIO.input(5) == False:
      print("Press S2")
    elif  GPIO.input(6) == False:
      print("Press S3")
    else:
      print("None")
```

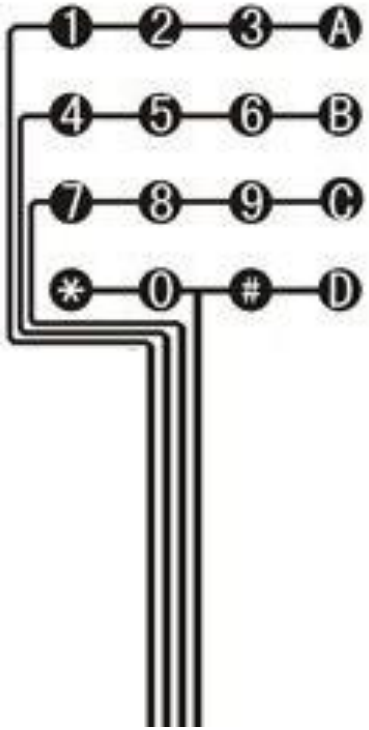
เปลี่ยนจากต่อ **gnd** มาต่อเข้า **GPIO2**

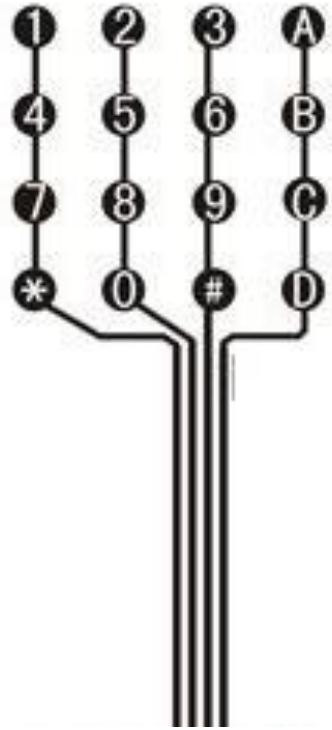
# Multiple Switch => Matrix Switch



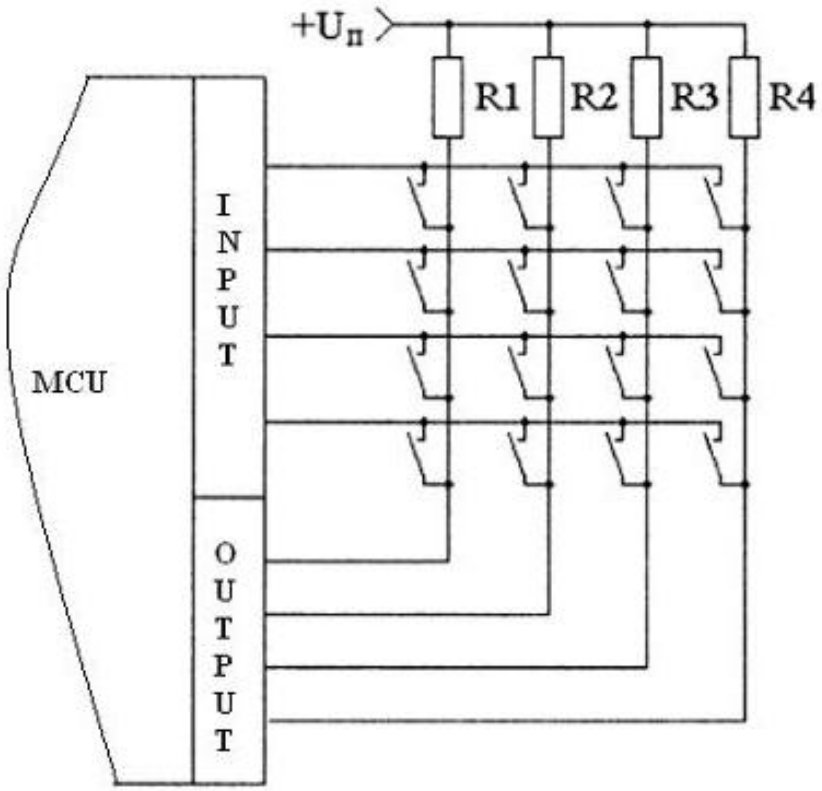
# 4. Keypad


1
2
3
A
4
5
6
B
7
8
9
C
*
0
#
D
Pin 1
Pin 8
Pins
1-4
are the rows
Pins
5-8
are the columns


1
2
3
A
4
5
6
B
7
8
9
C
*
0
#
D


1
2
3
A
4
5
6
B
7
8
9
C
*
0
#
D

# Keypad

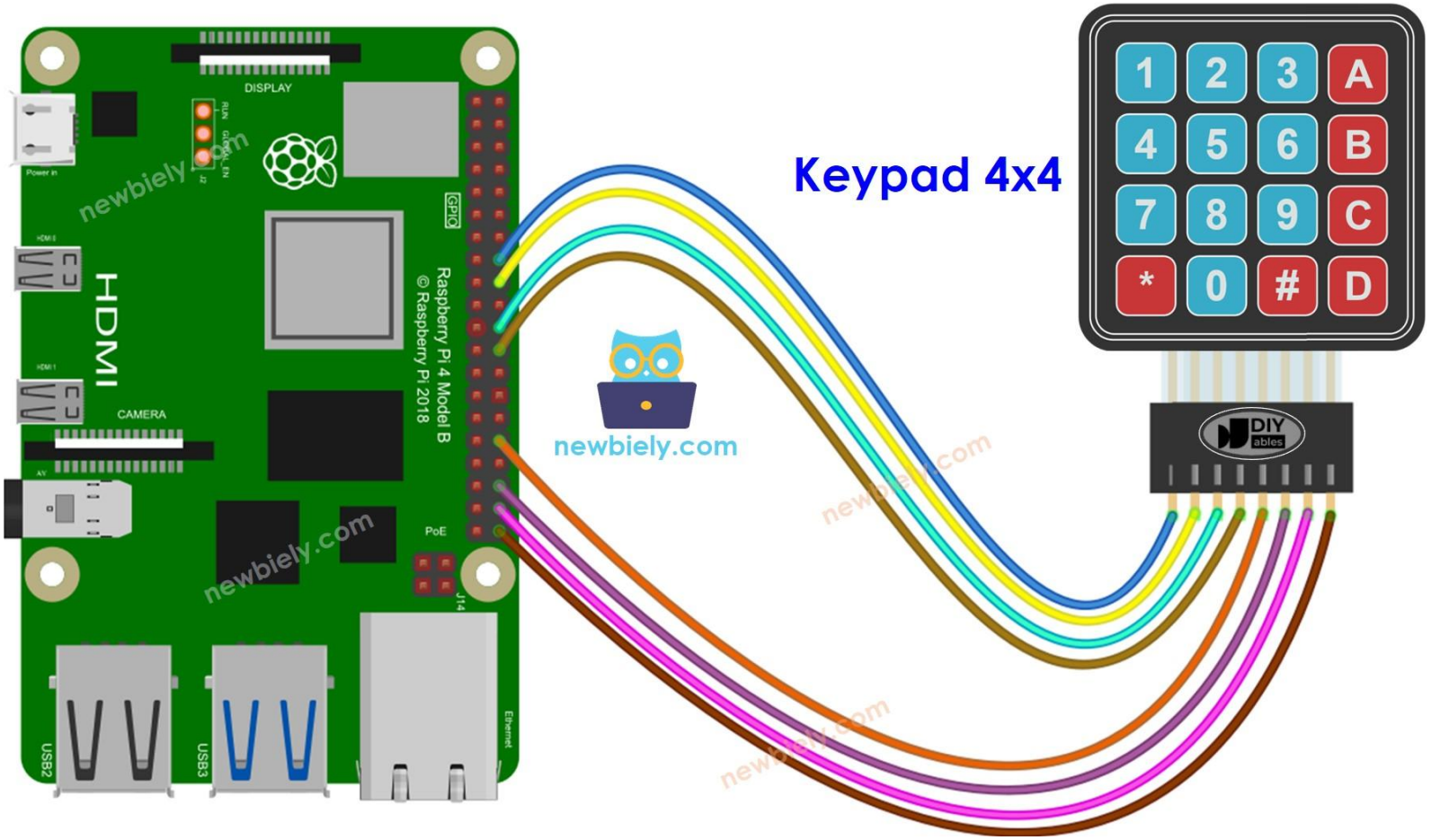



การต่อ **keypad** กับ **Raspi**

แต่ละแถวจะต่อกับ GPIO pin Input ของ Raspi

แต่ละคอลัมน์ จะต่อกับ GPIO pin Output ของ Raspi

# Keypad




Copy Code จาก URL ข้างล่างนี้

ที่มา https://newbiely.com/tutorials/raspberry-pi/raspberry-pi-keypad

```
import RPi.GPIO as GPIO
import time

# Define keypad layout
KEYPAD = [
    [1, 2, 3, 'A'],
    [4, 5, 6, 'B'],
    [7, 8, 9, 'C'],
    ['*', 0, '#', 'D']
]

# Define GPIO pins for rows and columns
ROWS = [4, 17, 27, 22]
COLS = [5, 6, 13, 19]

# Initialize GPIO
GPIO.setmode(GPIO.BCM)

# Set up row pins as inputs with pull-up resistors
for row_pin in ROWS:
    GPIO.setup(row_pin, GPIO.IN, pull_up_down=GPIO.PUD_UP)

```

```
# Set up column pins as outputs
for col_pin in COLS:
    GPIO.setup(col_pin, GPIO.OUT)
    GPIO.output(col_pin, GPIO.HIGH)

def get_key():
    key = None

    # Scan each column
    for col_num, col_pin in enumerate(COLS):
        GPIO.output(col_pin, GPIO.LOW)

        # Check each row
        for row_num, row_pin in enumerate(ROWS):
            if GPIO.input(row_pin) == GPIO.LOW:
                key = KEYPAD[row_num][col_num]

                # Wait for key release
                while GPIO.input(row_pin) == GPIO.LOW:
                    time.sleep(0.05)

        GPIO.output(col_pin, GPIO.HIGH)

    return key
```

```
53
54 try:
55     while True:
56         pressed_key = get_key()
57
58         if pressed_key is not None:
59             print(f"Pressed: {pressed_key}")
60         time.sleep(0.1)
61
62 except KeyboardInterrupt:
63     GPIO.cleanup()
```

# แบบฝึกหัดที่ 1

ต่อ Keypad

เมื่อกด switch แต่ละตัว ให้ Color LED ติดคนละสี 7 สี

และเมื่อปล่อย switch ให้ Color LED ดับ

# Analog Signal



# 5. Analog Signal

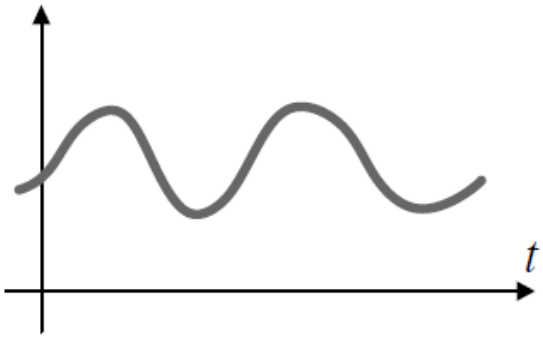
Analog เป็นภาษากรีก มาจากคำว่า analogous แปลว่า เหมือนกัน คล้ายคลึงกัน อุปมาเหมือน

**สัญญาณ** Analog คือ สัญญาณที่คงรูปเดิมเหมือนกับต้นฉบับ จะมีลักษณะรูปคลื่นเป็นสัญญาณที่ต่อเนื่อง

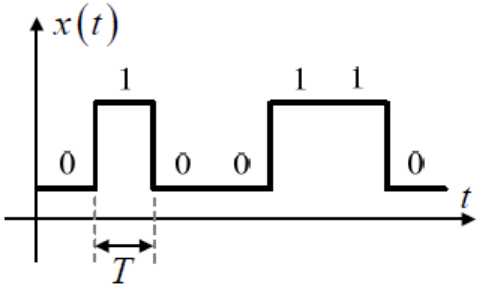
จากลักษณะทางกายภาพของสัญญาณ Analog แปลงมาเป็นสัญญาณทางไฟฟ้าเพื่อแสดงให้เห็นจริง (วัดได้) จะใช้อุปกรณ์ที่เรียกว่า Sensor

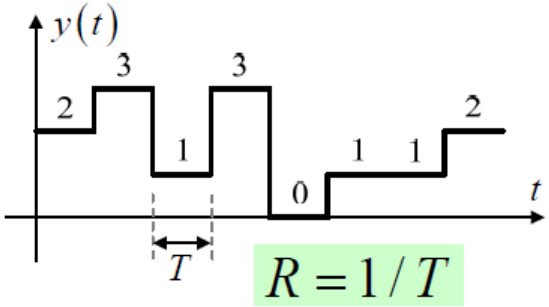
# สัญญาณแอนาล็อค (analog) & สัญญาณดิจิทัล (Digital)

- **สัญญาณแอนาล็อค** คือสัญญาณที่ขนาด (แอมพลิจูด) มีการเปลี่ยนแปลงอย่างต่อเนื่องทางเวลา
  - สัญญาณแอนะล็อกทุกสัญญาณเป็นสัญญาณที่ต่อเนื่องทางเวลา
  - นั่นคือ ค่าของสัญญาณแอนะล็อกจะเป็นสมาชิกของเซตจำนวนจริ
  - สัญญาณในธรรมชาติ เป็น สัญญาณแอนาล็อค
  - Analog มาจากคำว่า **analogous แปลว่า** เหมือนกัน, ค

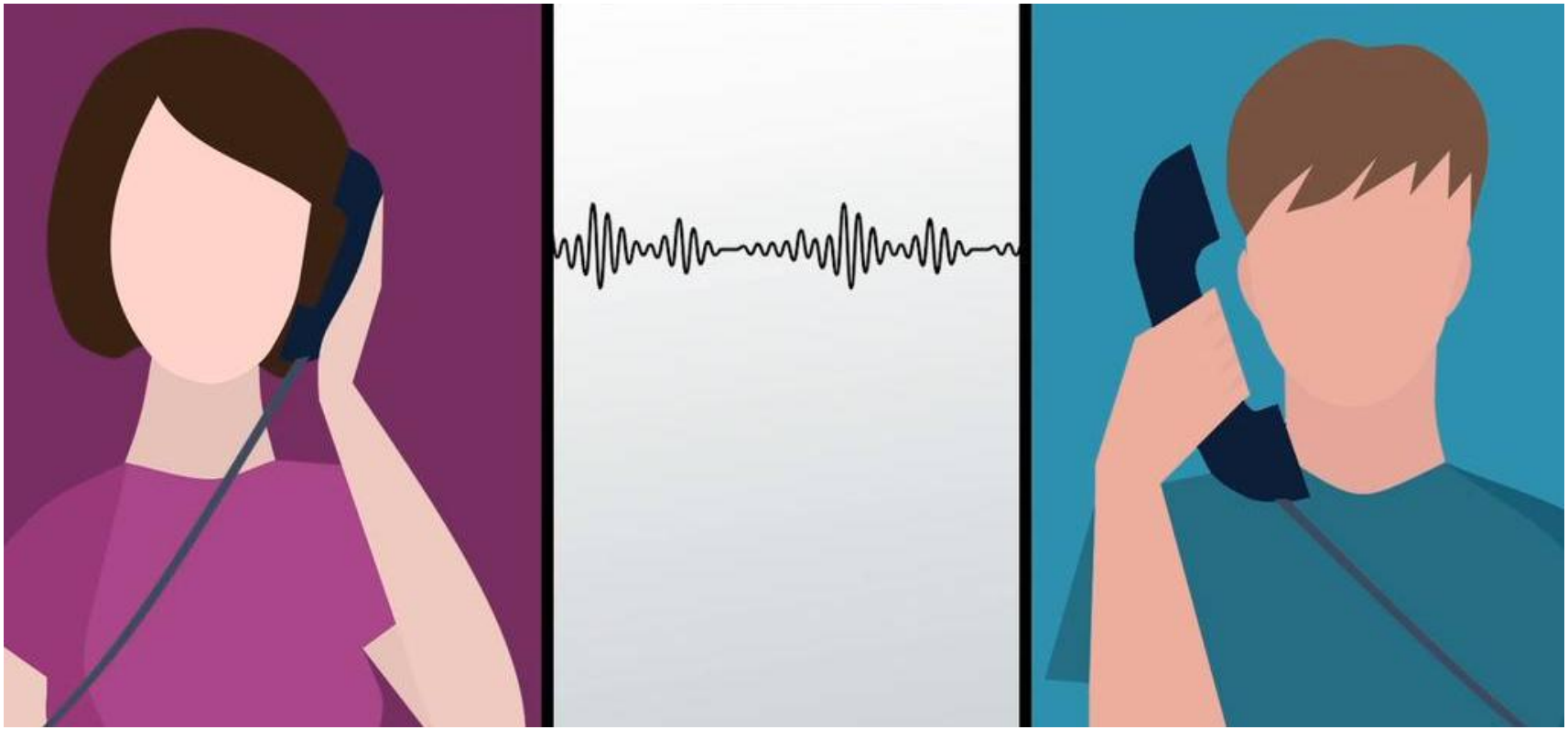



- **สัญญาณดิจิทัล** คือสัญญาณที่ขนาด (แอมพลิจูด) มีค่าระดับที่แน่นอน
  - นั่นคือ ค่าของสัญญาณดิจิทัล จะอยู่ภายในเซตจำกัด



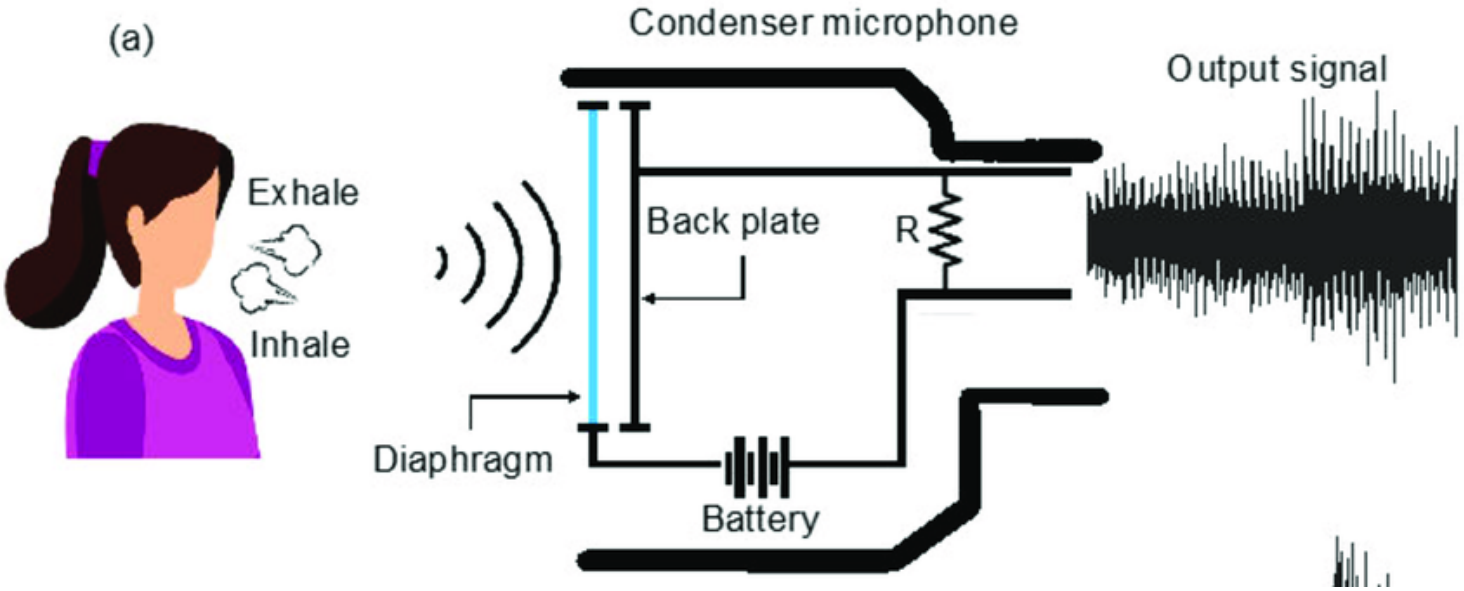



# ตัวอย่างสัญญาณ **Analog**



# ตัวอย่างสัญญาณ Analog



# ตัวอย่างสัญญาณ **Analog**

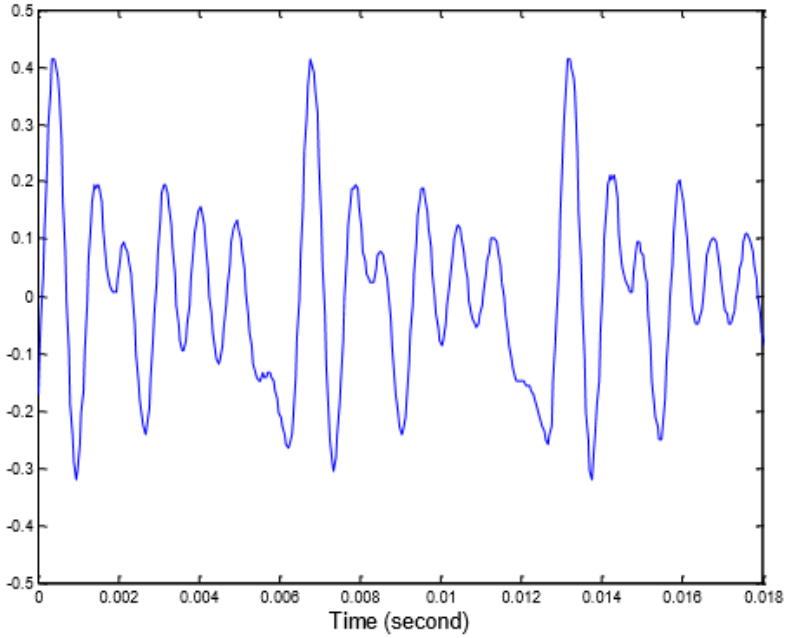
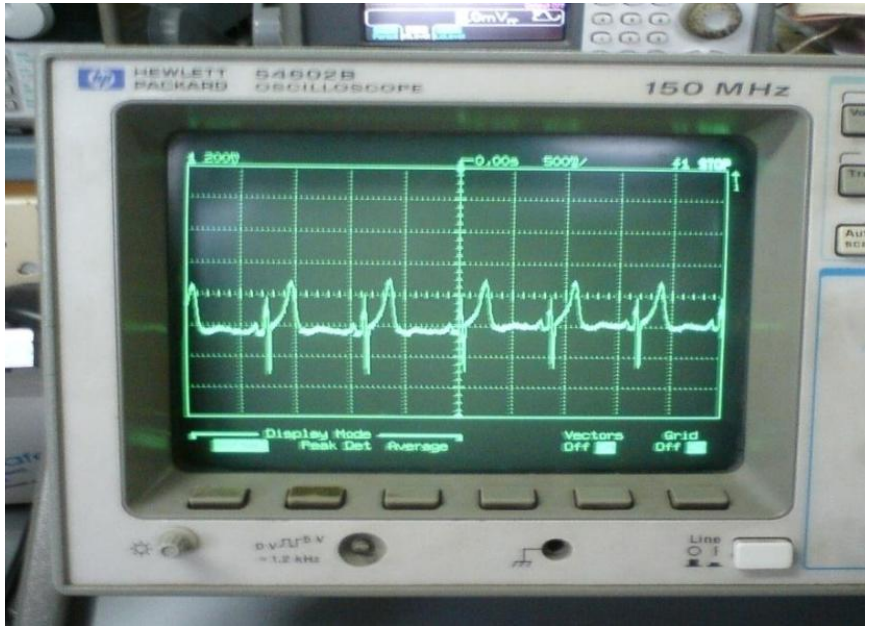


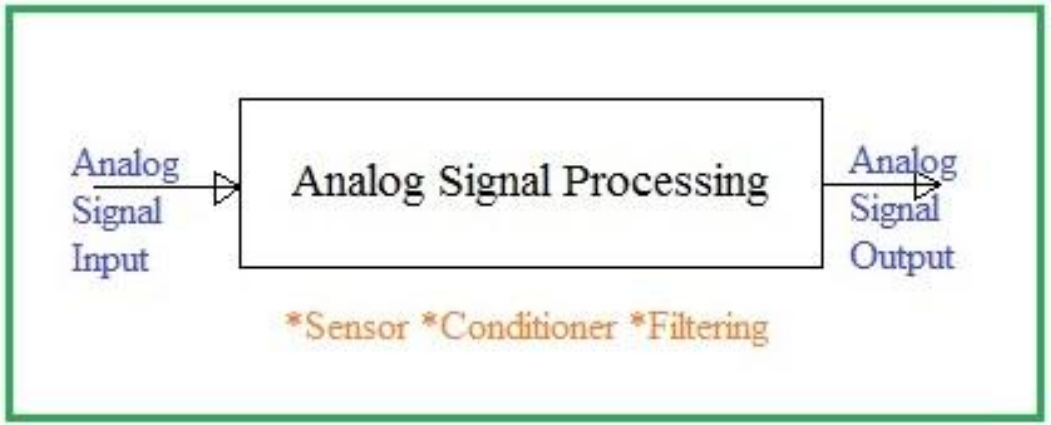

**Figure 2.** Example of speech signal

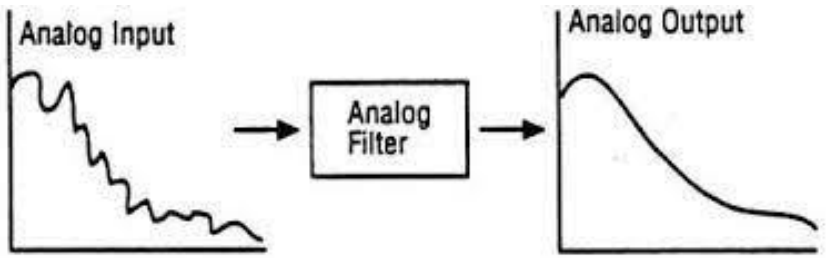


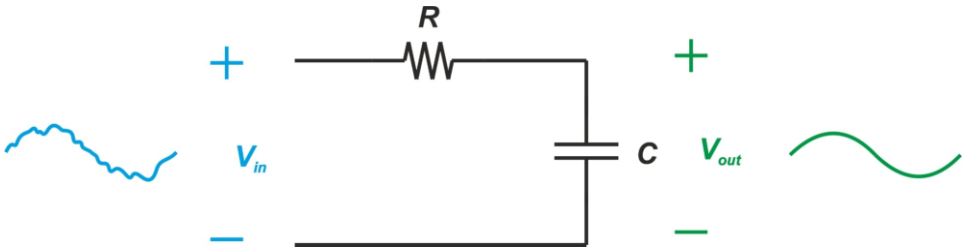

Example of ECG Signal

# 6. Analog Signal Processing

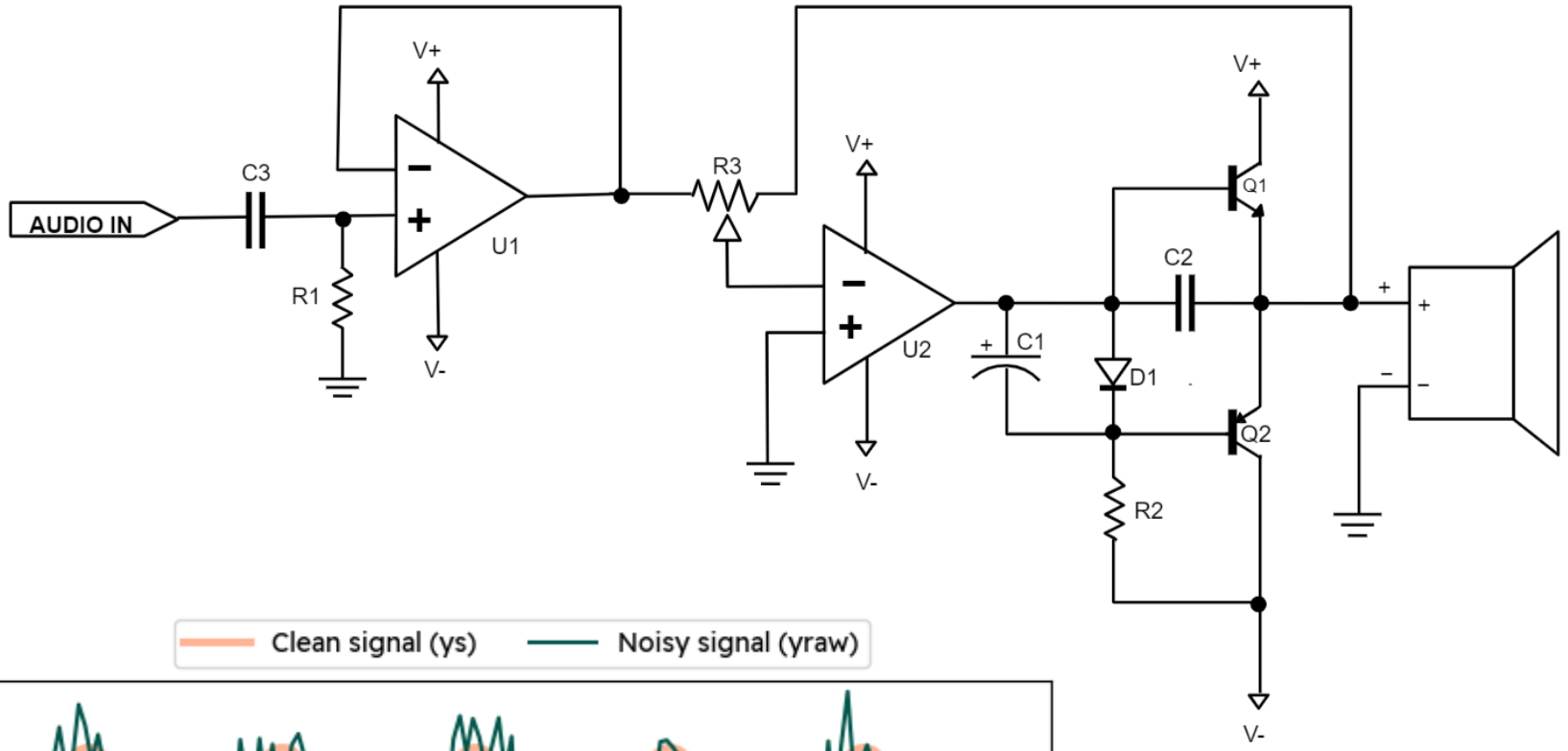
การประมวลผลสัญญาณ (signal processing) เช่น การขยายสัญญาณ (amplifier)
การกำจัดสัญญาณรบกวน (filter) การผสมสัญญาณ (Mix)

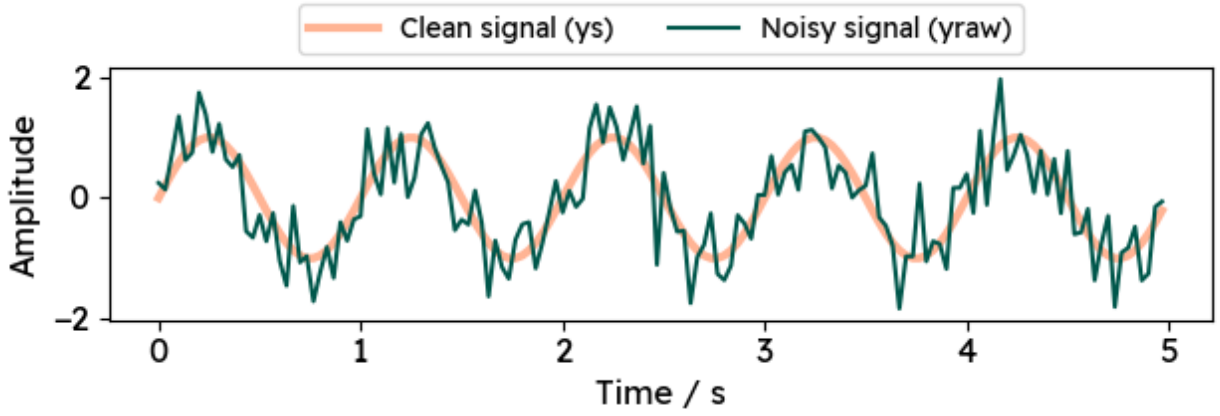


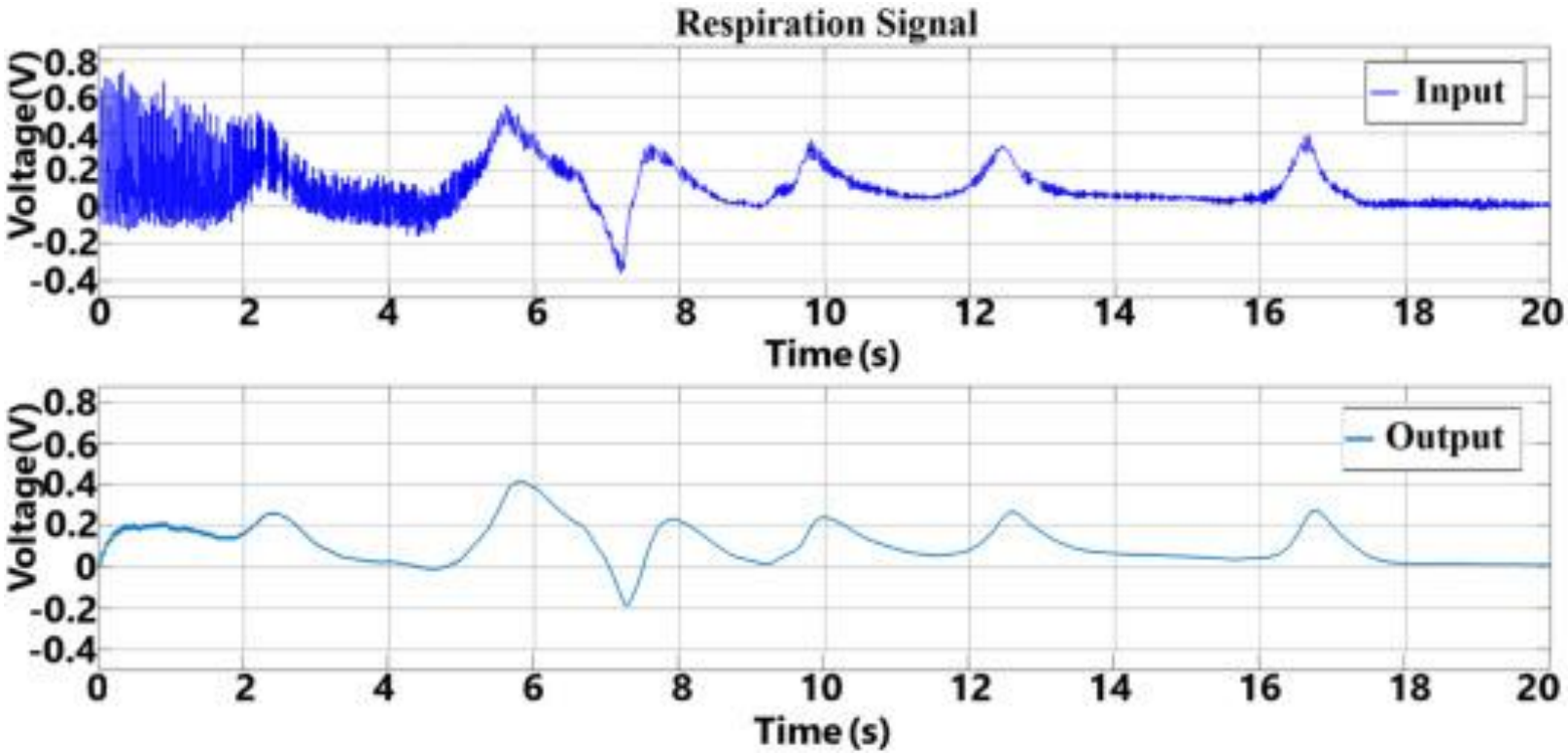
# ตัวอย่าง Analog Signal Processing circuit




ตัวอย่างวงจร **filter**

# ตัวอย่าง Analog Signal Processing

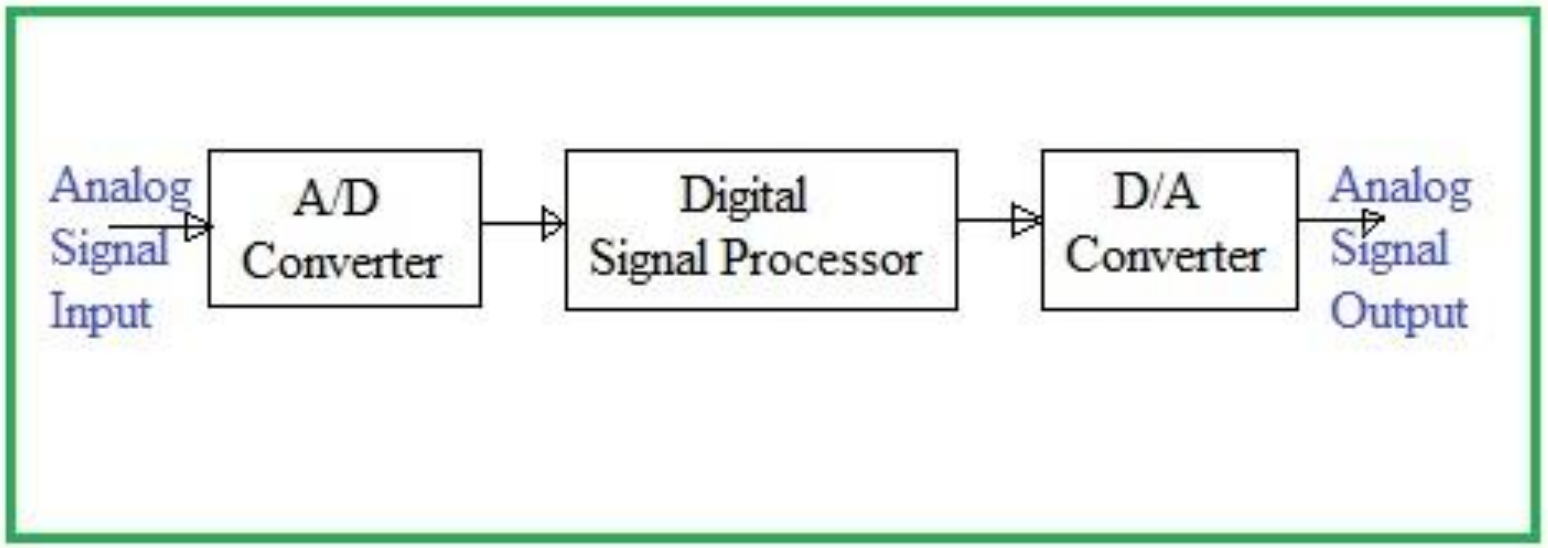



ตัวอย่าง การ **filter** สัญญาณ

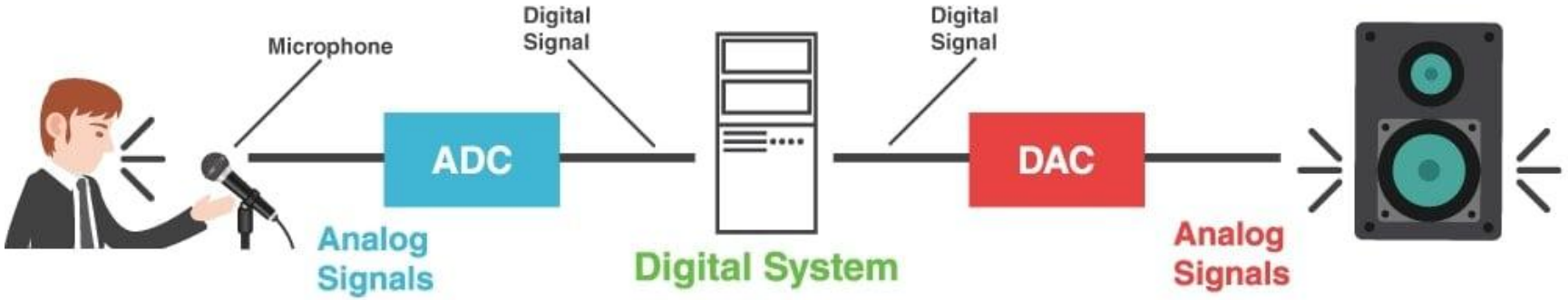
# 7. Digital Signal Processing

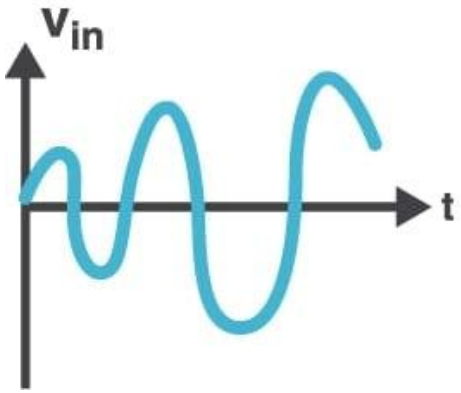
ในอดีต การประมวลสัญญาณ นิยมต่อวงจร ซึ่งมีข้อจำกัดคือ วงจรมีขนาดใหญ่ เสียเวลาในการต่อ การผลิต และต่อมาเทคโนโลยี chip มีการพัฒนามากขึ้น สามารถทำงานได้ไวขึ้น จึงมีแนวคิด นำสัญญาณมาแปลงเป็นสัญญาณดิจิทัล แล้วทำสัญญาณดิจิทัลนั้น มาประมวลด้วยการเขียนโปรแกรม จึงเป็นที่มาของ Digital Signal Processing

มี Block Diagram หลักๆ ดังรูป



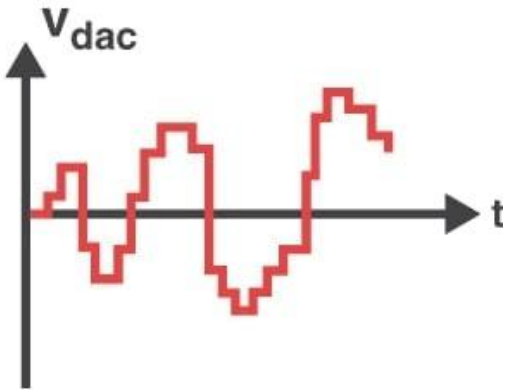
# Digital Signal Processing



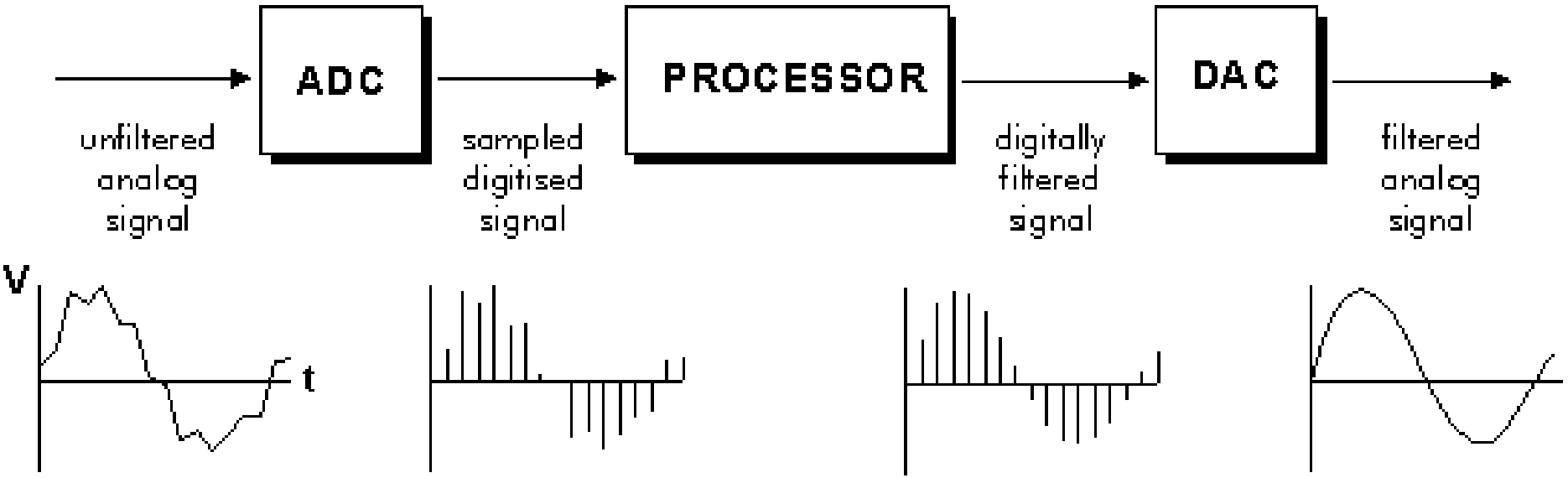









# Digital Signal Processing




DSP

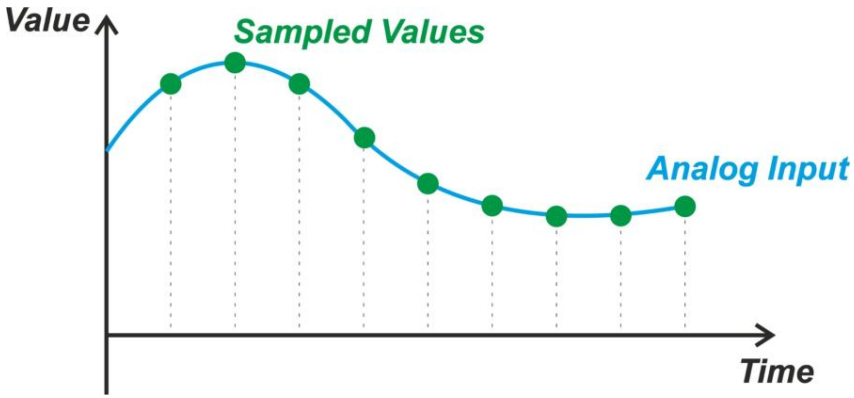
**ข้อดี** คือ ประมวลผลด้วยการเขียนโปรแกรม วงจรไม่สลับซับซ้อน สามารถทำงานได้แม่นยำกว่า

**ข้อเสีย** คือ ต้องมีความรู้ในการเขียนโปรแกรม มีความเข้าใจในสมการคณิตศาสตร์ และ Chip อาจจะมีราคาแพงกว่าการต่อวงจรอนาล็อค

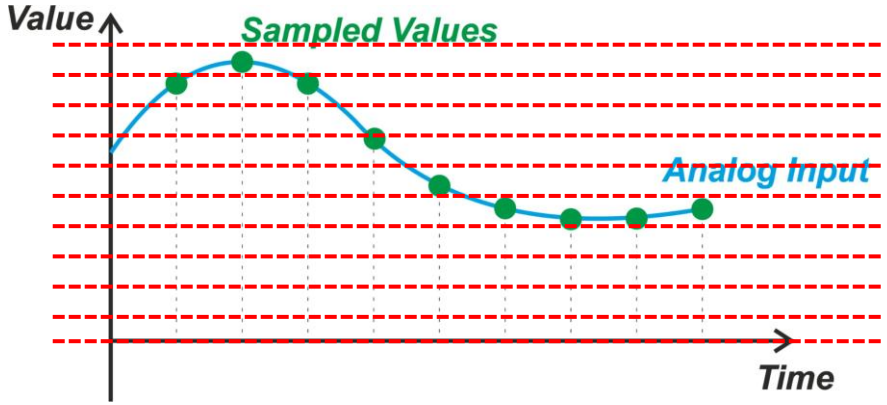
# 7. Analog to Digital

กระบวนการแปลงสัญญาณ Analog ไปเป็นสัญญาณ Digital จะมีขั้นตอนหลักๆ คือ

- การ Sampling ( ต้อง Sampling ด้วยความถี่ ที่มากกว่า 2 เท่าของความถี่สูงสุดของสัญญาณที่เข้ามา $f_s \geq 2f_m$ )
- การ Encoding หรือ การ Quantize (กำหนดโดยค่า Resolution)

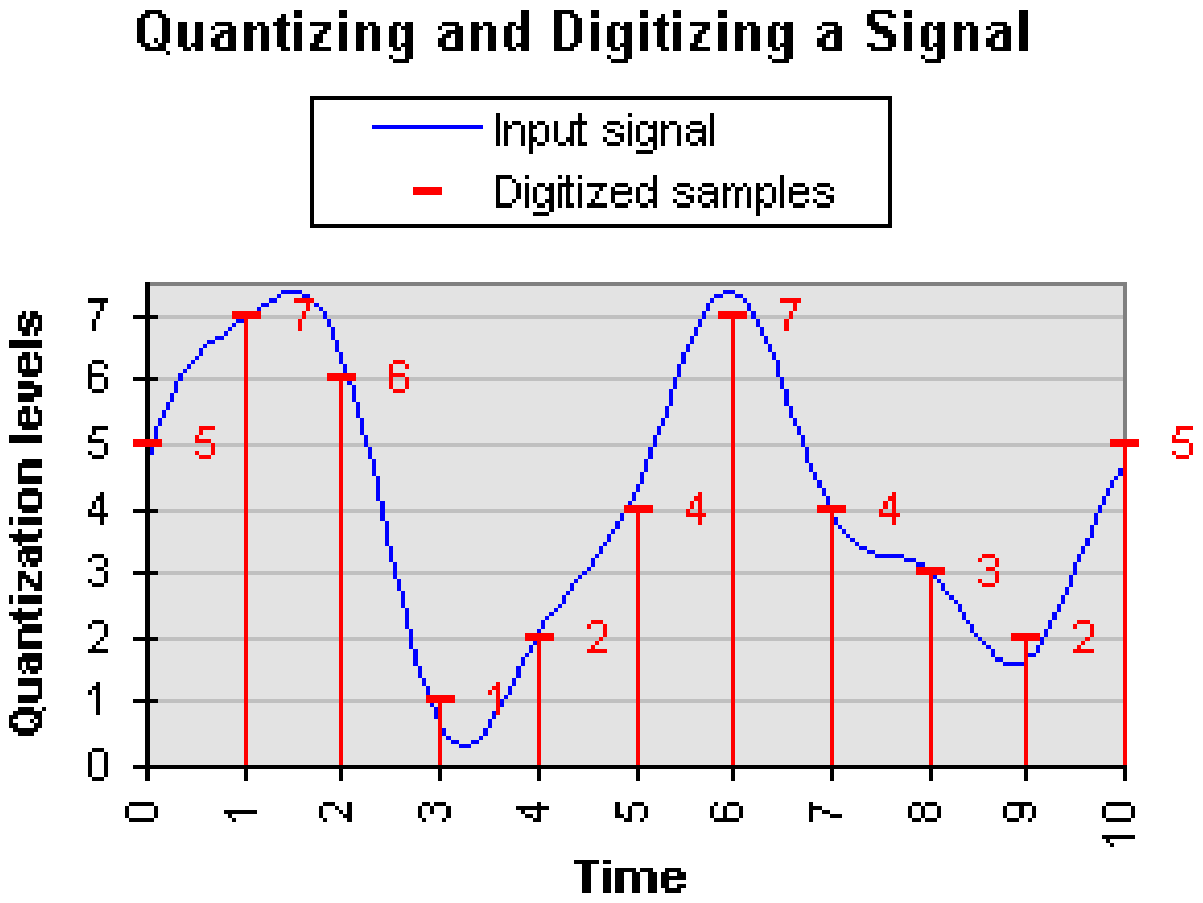



Sampling




Quantize

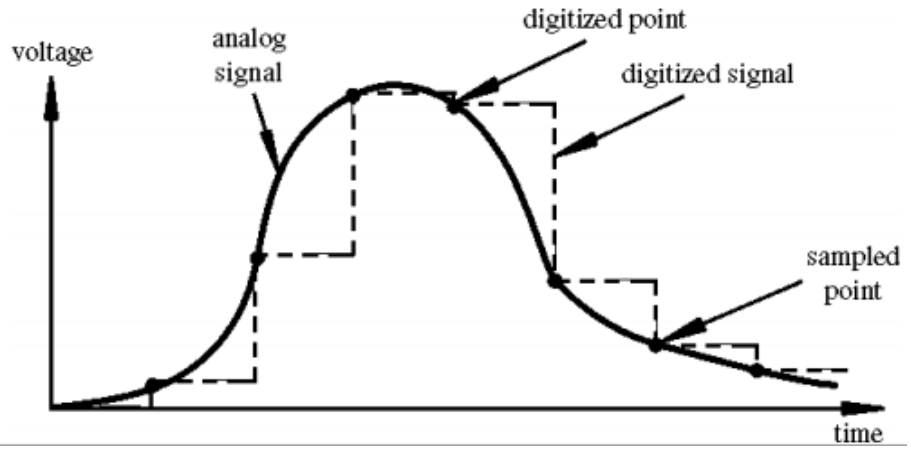
# Analog to Digital



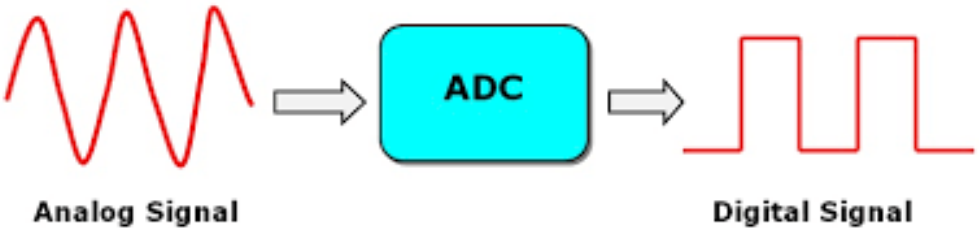

ถ้า **sample** ไม่ตรงระดับ จะเกิด **Quantize Noise**

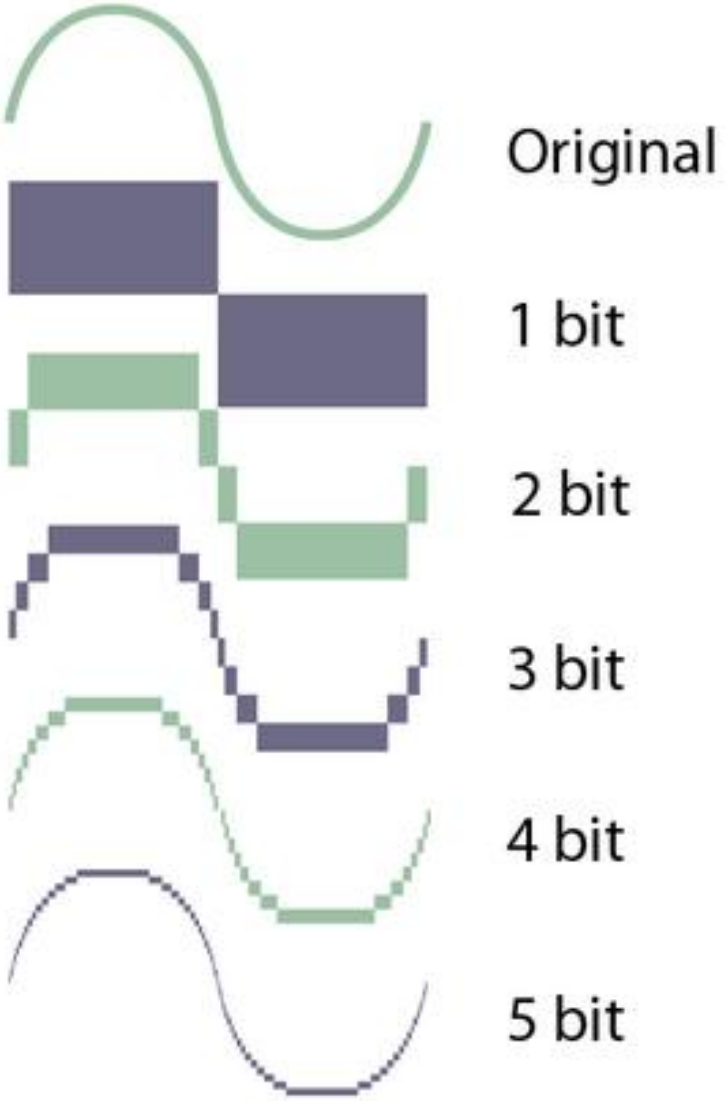
# Analog to Digital

- Many states, not just two (HIGH/LOW)
- Number of states (or values, or "bins") is *resolution*
- Common computer resolutions:
  - 8-bit = 256 values
  - 16-bit = 65,536 values
  - 32-bit = 4,294,967,296 values

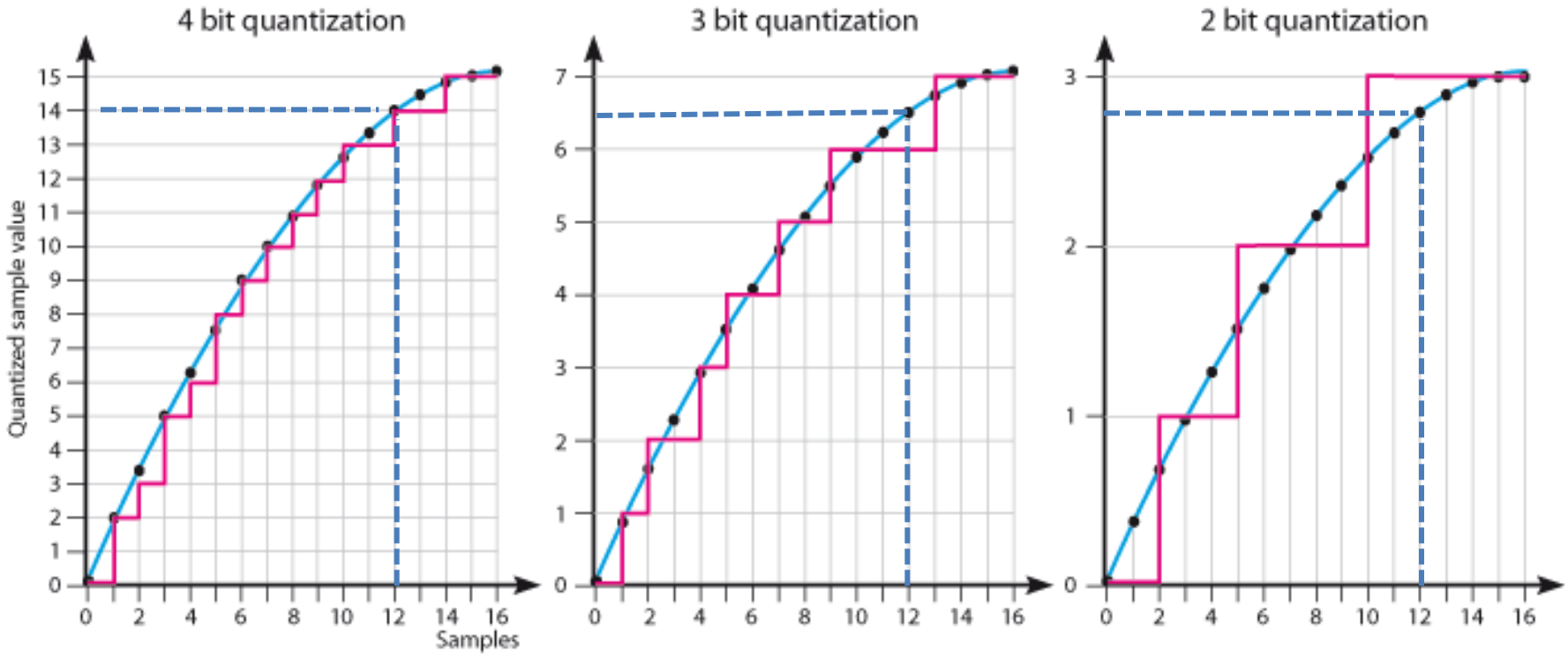


# 8. Resolution



# Resolution




พิจารณา ตัวอย่าง sample ที่ 12

4bit

อยู่ที่ระดับที่ 14 = 1110

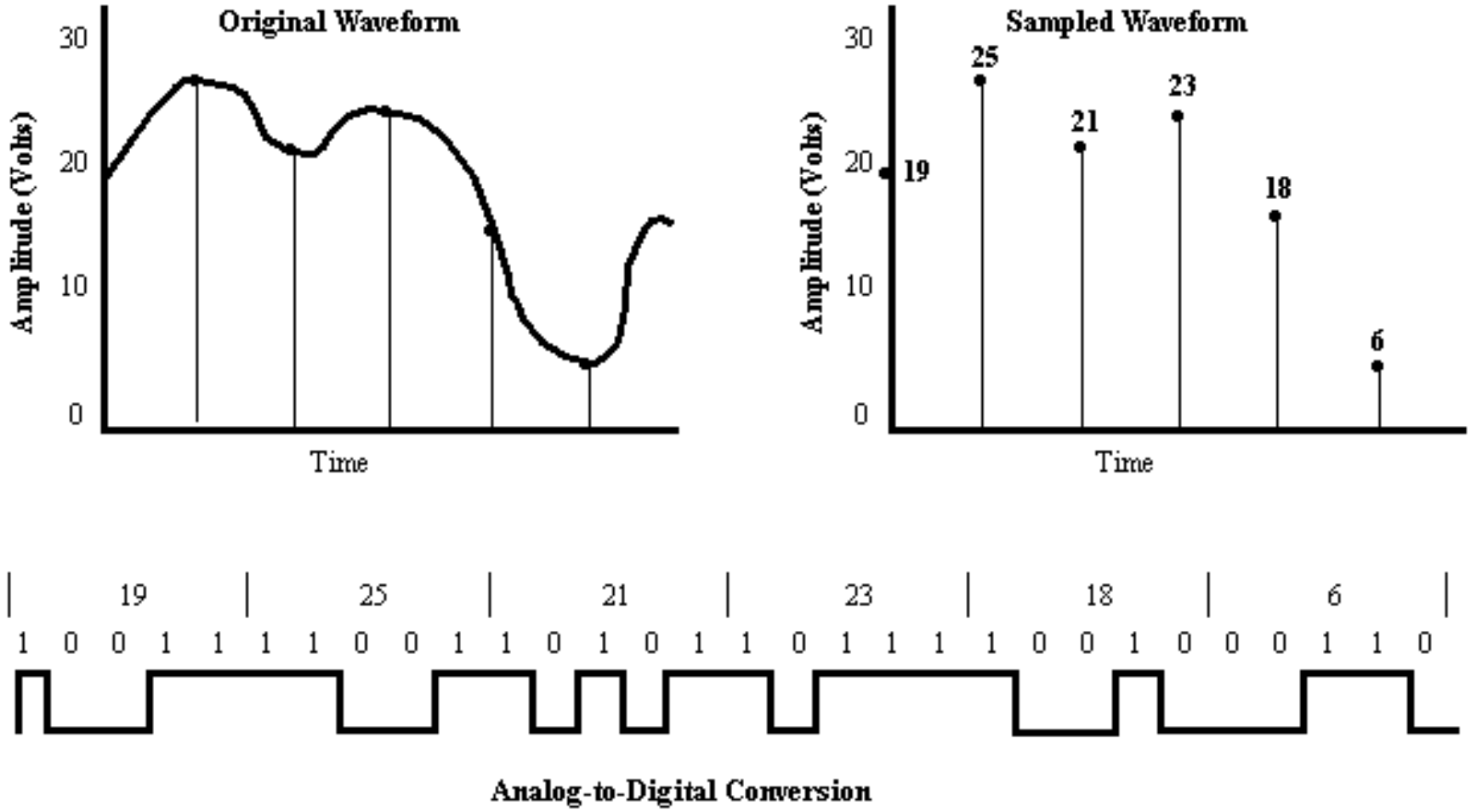
3 bit

อยู่ที่ระดับที่ 7 = 111

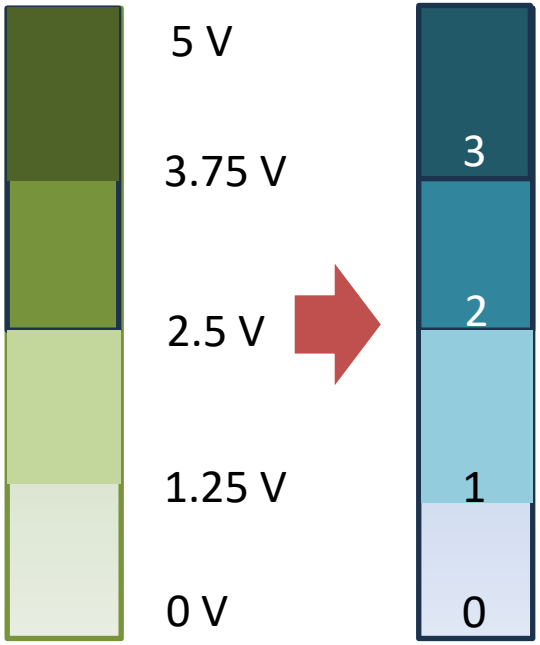
2 bit

อยู่ที่ระดับที่ 3 = 11

# Resolution ตัวอย่าง 5 bit



# **ตัวอย่าง** การคำนวณหาค่า digital จากแรงดัน analog ที่วัดได้




Analog input

ADC Value
ที่ MCU อ่านได้

สมมุติว่า ADC ขนาด 2 bit
จะให้ค่าดิจิทัล มีค่า 0 , 1, 2, 3

$$Resolution = \frac{V_{ref}}{2^{N\ bits}\ \text{ระดับ}} = \frac{5\ volt}{4\ \text{ระดับ}}$$

= 1.25 volt/ระดับ

```
0      1.25     2.50     3.75      5V
|--------|--------|--------|--------|
   00       01       10       11
```

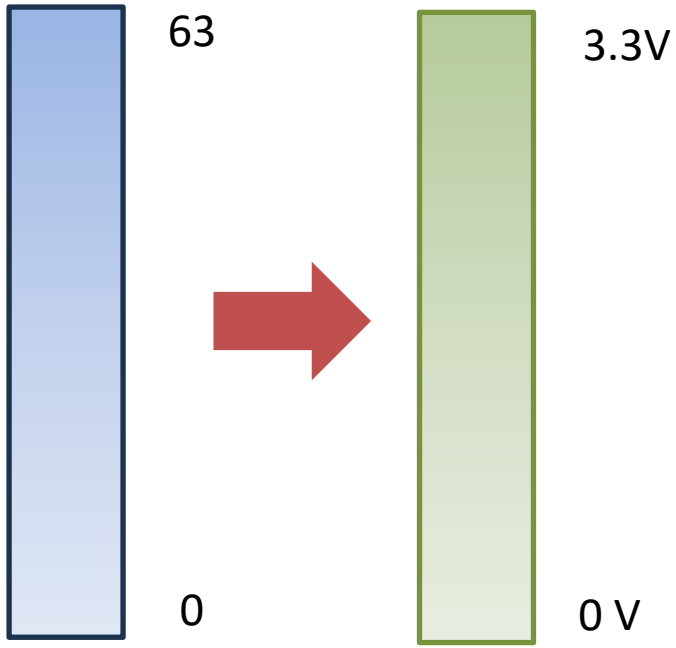
จะได้ว่า

$$Digital\ Value = v_{in} \times \frac{2^{N\ bits}}{V_{ref}}$$

โดยค่า Digital ที่ได้จะอยู่ในช่วง

$$0, \ldots, 2^{N\ bits} - 1$$

# **ตัวอย่าง** การคำนวณหาค่าแรงดัน analog จากค่า digital ที่อ่านได้




สมมุติว่า ADC อ่านค่าได้ 20

จากสมการหน้าที่แล้ว ย้ายข้างสมการ จะได้ว่า

$$v_{in} = Digital\ Value \times \frac{V_{ref}}{2^{N\ bits}}$$

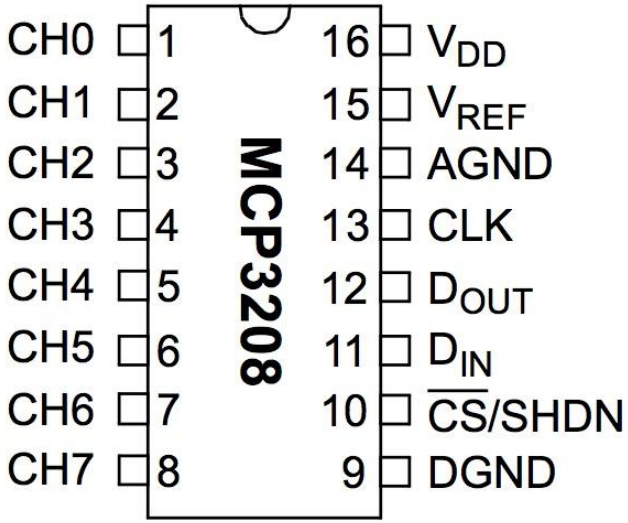
$$v_{in} = 20 \times \frac{3.3}{64}\ volt$$

$$= 1.03125 \text{ volt}$$

สมมุติว่า ADC มี Resolution ขนาด 6 bit
จะให้ค่าดิจิทัล มีค่า 0-63
และแรงดัน input ที่ป้อนเข้ามา มีค่า 0 - 3.3 Volt

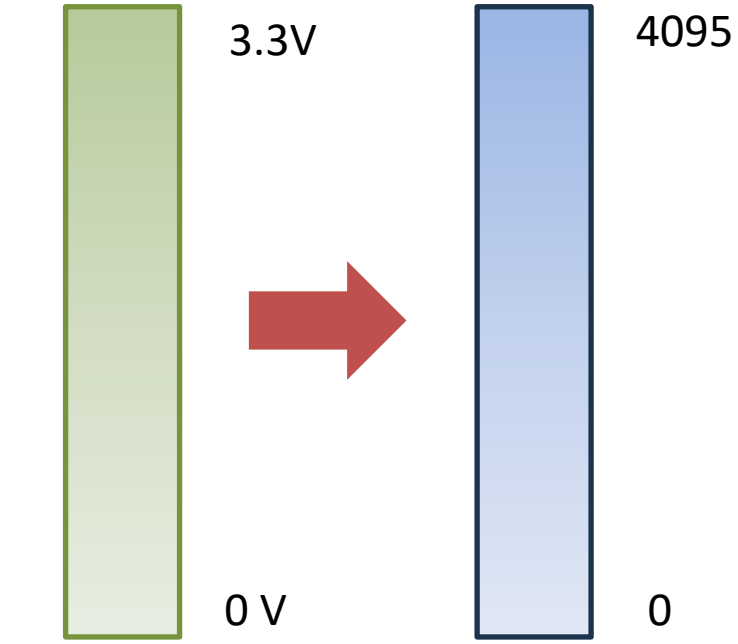
# 9. IC Analog to Digital : MCP3208




- มีค่า Resolution 12 bits ( 0 – 4095)
- มี Analog Input ทั้งหมด 8 Channels
- สามารถ Sampling ได้ประมาณ 100 kSamp/s ที่ $V_{DD}$ = 5V (ค่ารวมทั้ง 8 ช่อง ถ้าใช้มากกว่า 1 ช่อง ความเร็วก็ต้องหารด้วยจำนวนช่องที่ใช้งานด้วย)
- ใช้การเชื่อมต่อกับ Microcontroller แบบ **Serial Peripheral Interface (SPI)** ซึ่งประกอบด้วยขา CLK, DOUT, DIN, CS, DGND
- $V_{REF}$ คือ ค่าแรงดันสูงสุดที่ป้อนเข้าได้ แต่ต้องมีค่าไม่มากกว่าค่า $V_{DD}$

# กรณี Analog Input ($V_{REF}$ = 3.3V)



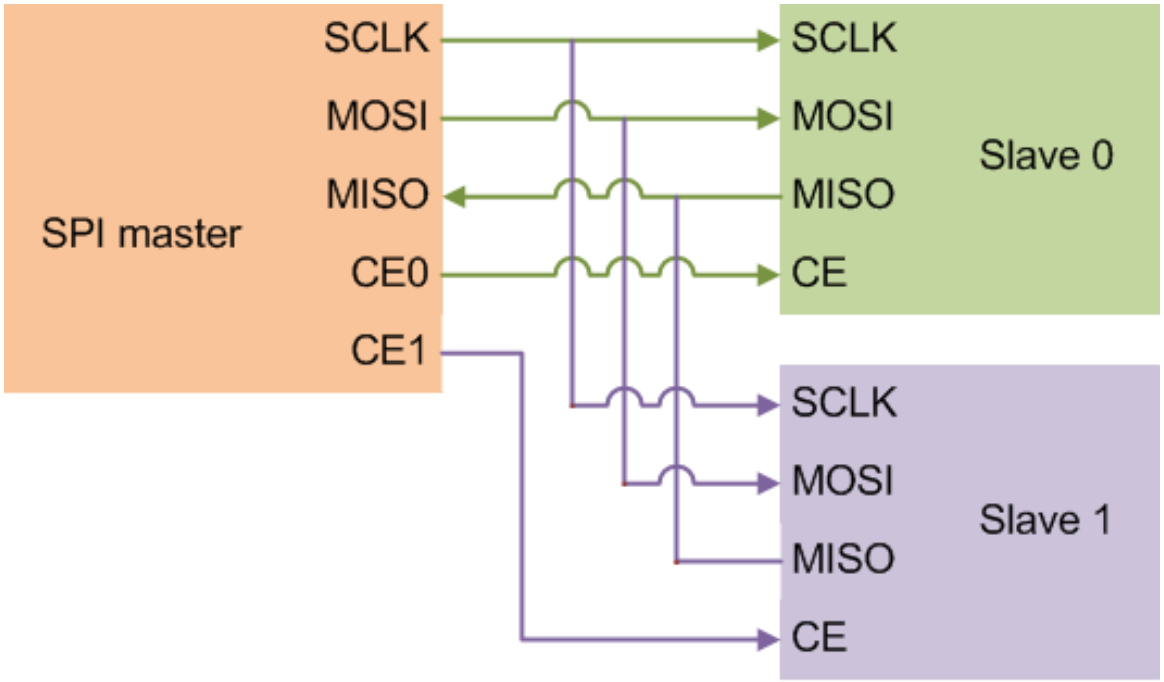

$$Digital\ Value = v_{in} \times \frac{4096}{3.3}$$

0 V => 0000 0000 0000 => 0
0.8 mV => 0000 0000 0001 => 1
1.61 mV => 0000 0000 0010 => 2
2.41 mV => 0000 0000 0011 => 3
…
3.2983 V => 1111 1111 1110 => 4094
3.2991 V => 1111 1111 1111 => 4095

แต่ละระดับมีค่าเท่ากับ 3.3 V / 4096 ระดับ = 0.8056 mV

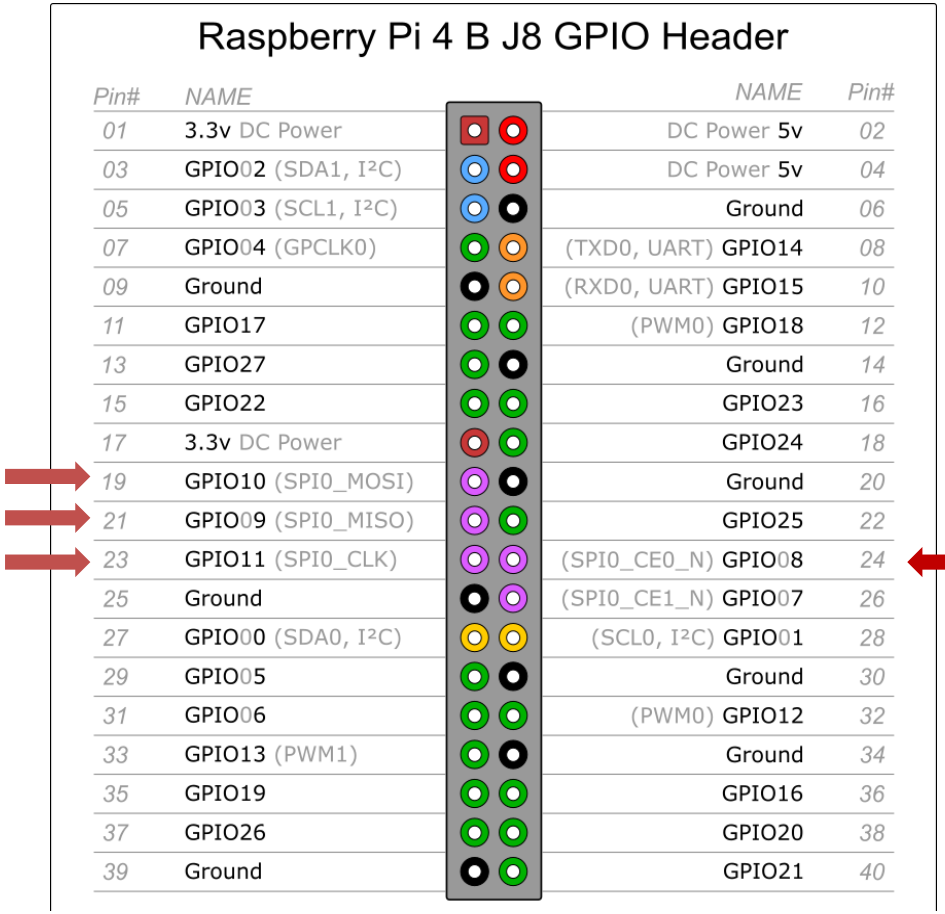
# SPI : Serial Peripheral Interface

SPI คือวิธีการสื่อสารอนุกรมแบบ Synchronous รูปแบบหนึ่ง ซึ่งทำงานในรูปแบบที่ให้อุปกรณ์ตัวหนึ่งทำหน้าที่เป็น Master ในขณะที่อีกตัวหนึ่งทำหน้าที่เป็น Slave และสามารถส่งข้อมูลในโหมด Full-duplex นั่นหมายความว่าสามารถส่งข้อมูลหากันได้ระหว่าง Master และ Slave ได้อย่างต่อเนื่อง ไป-กลับ ได้พร้อมกัน



# SPI : Serial Peripheral Interface

- การสื่อสารระหว่าง Master กับ Slave จะใช้ Port ของอุปกรณ์ประกอบด้วย
  - **MISO** (Master In Slave Out) คือ ออกจาก Slave มาเข้า Master
  - **MOSI** (Master Out Slave In) คือ ออกจาก Master มาเข้า Slave
  - **SCK** (Serial Clock) ขานี้เป็นขาที่ Master ใช้กำหนดจังหวะในการส่งข้อมูล
  - **Chip Select** ขานี้ไว้ใช้เลือกอุปกรณ์ที่เป็น Slave ซึ่งอาจจะมีได้มากกว่า 1 ตัว



# Library : SPIdev

https://github.com/doceme/py-spidev

คือ **Python library** สำหรับติดต่อสื่อสารกับอุปกรณ์ **SPI (Serial Peripheral Interface)** บนระบบที่ใช้ **Linux** ผ่าน **spidev driver**

**Settings**

```
import spidev
spi = spidev.SpiDev()
```

**Methods** หลัก

`open(bus, device)` เปิดการเชื่อมต่อ SPI ผ่านอุปกรณ์,

`readbytes(n)` อ่านข้อมูล n ไบต์จาก SPI (Half-duplex)

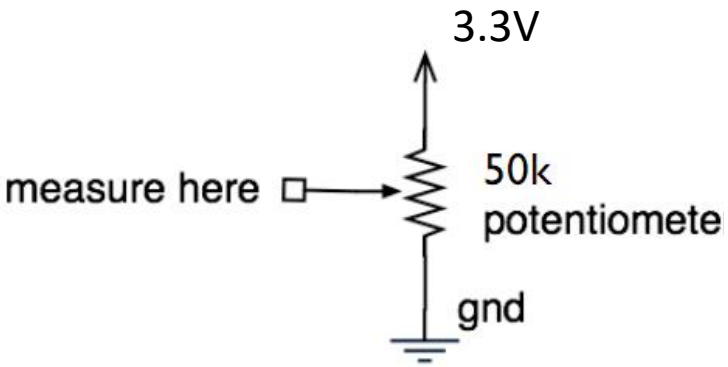
`writebytes(list_of_values)` ส่งข้อมูลแบบ Half-duplex

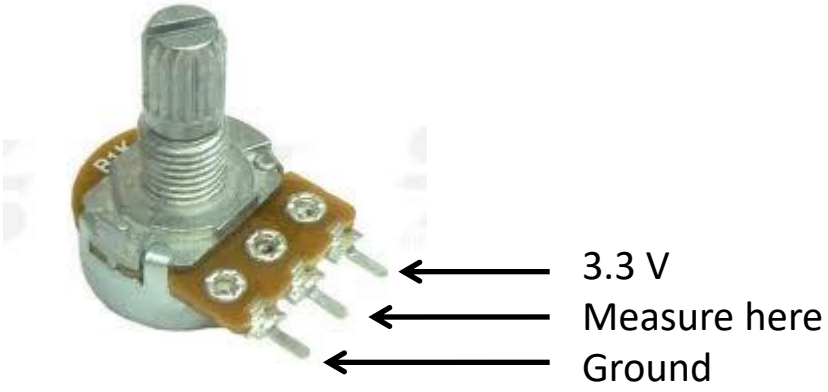
`xfer2(list of values[, speed_hz, delay_usec, bits_per_word])`

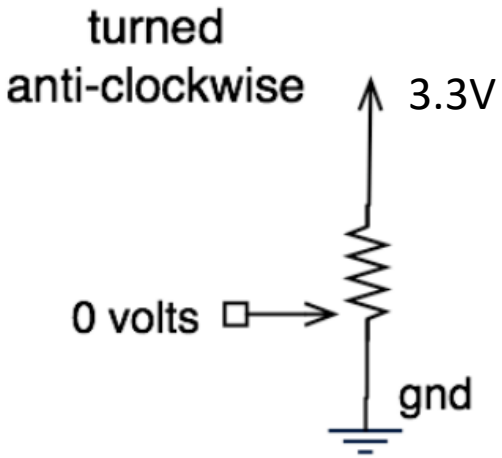
ส่งและรับข้อมูลแบบ full-duplex (list เดียวส่งออกและรับค่าตอบกลับ)

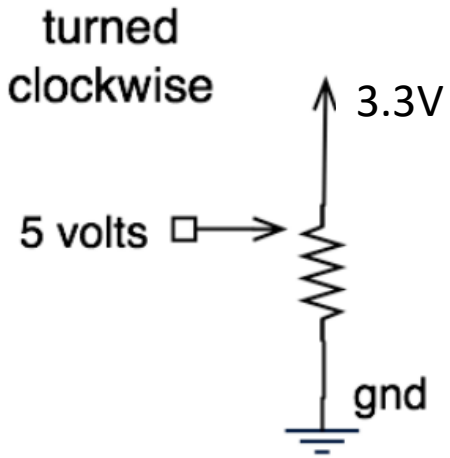
`close()` ปิดการเชื่อมต่อกับอุปกรณ์ SPI

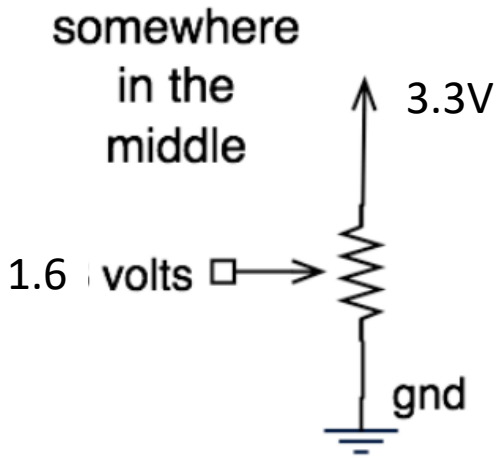
# How to make a varying voltage ?



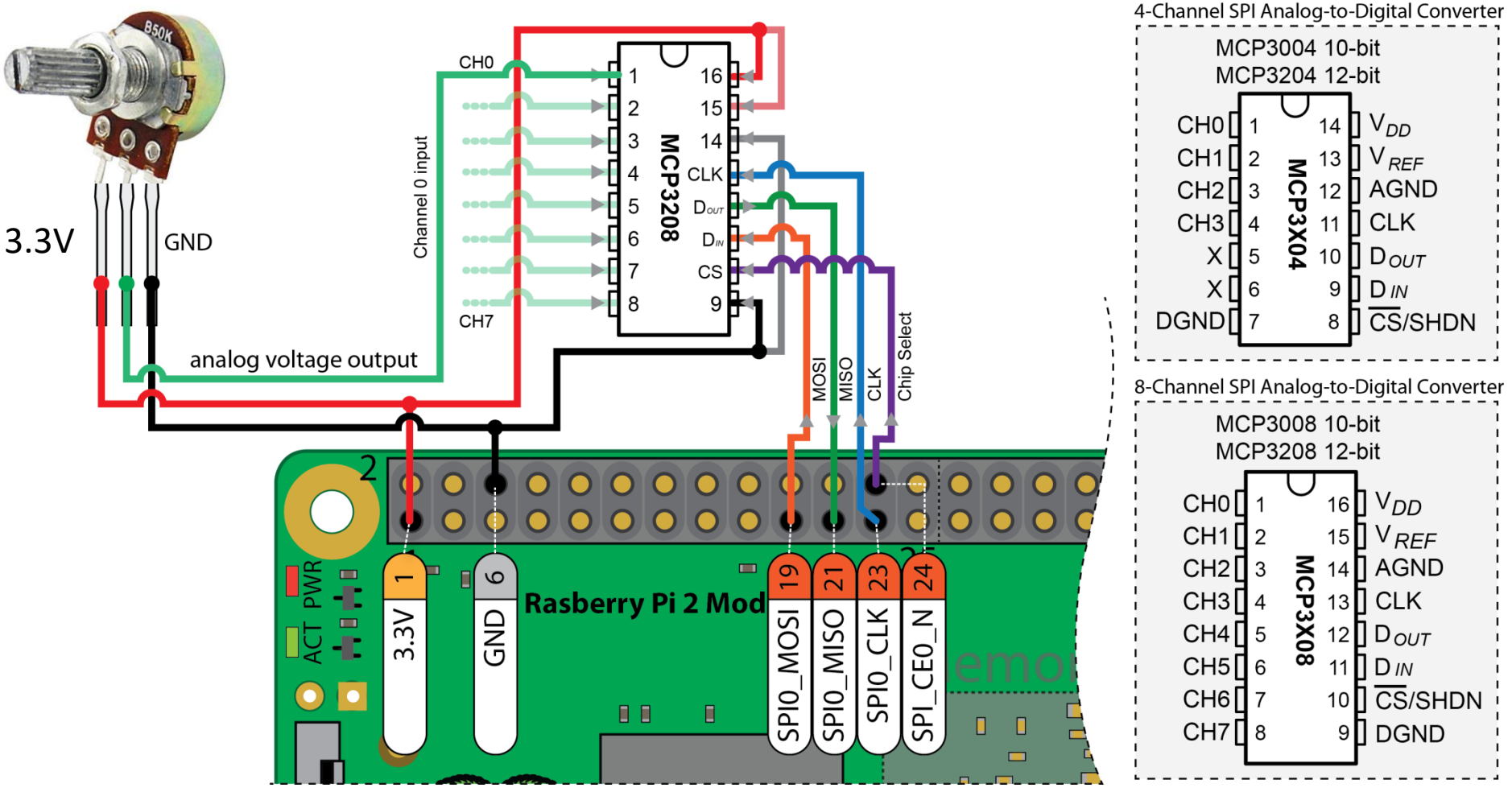



# ตัวอย่าง การต่อวงจร : MCP3208



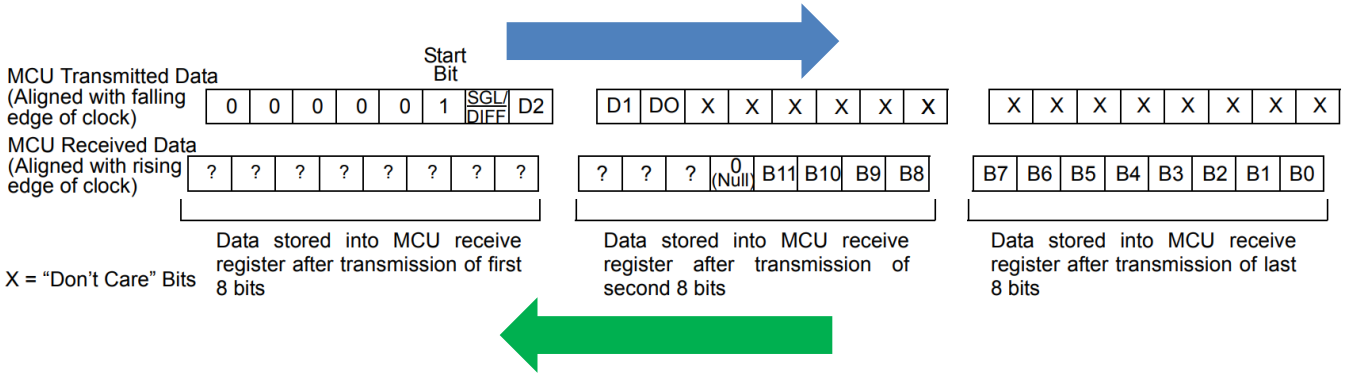
# การสื่อสารกับ MCP3208

- MCP3208 ใช้การเชื่อมต่อกับ Microcontroller แบบ Serial Peripheral Interface (SPI) ซึ่งประกอบด้วยขา CLK, Dout, Din, CS, DGND

  - ขา MOSI ของ Raspberry pi ต่อเข้ากับขา Din ของ MCP3208
  - ขา MISO ของ Raspberry pi ต่อเข้ากับขา Dout ของ MCP3208
  - ขา CLK ของ Raspberry pi ต่อเข้ากับขา CLK ของ MCP3208
  - ขา CS0 ของ Raspberry pi ต่อเข้ากับขา CS ของ MCP3208

- Raspberry pi ส่งคำสั่ง ไปยัง MCP3208 ด้วยขนาด 3 byte
- จากนั้น MCP3208 ส่งค่าข้อมูลดิจิทัลที่แปลงได้ กลับมาที่ Rasberry Pi ด้วยขนาด 3 byte








# การสื่อสารกับ MCP3208

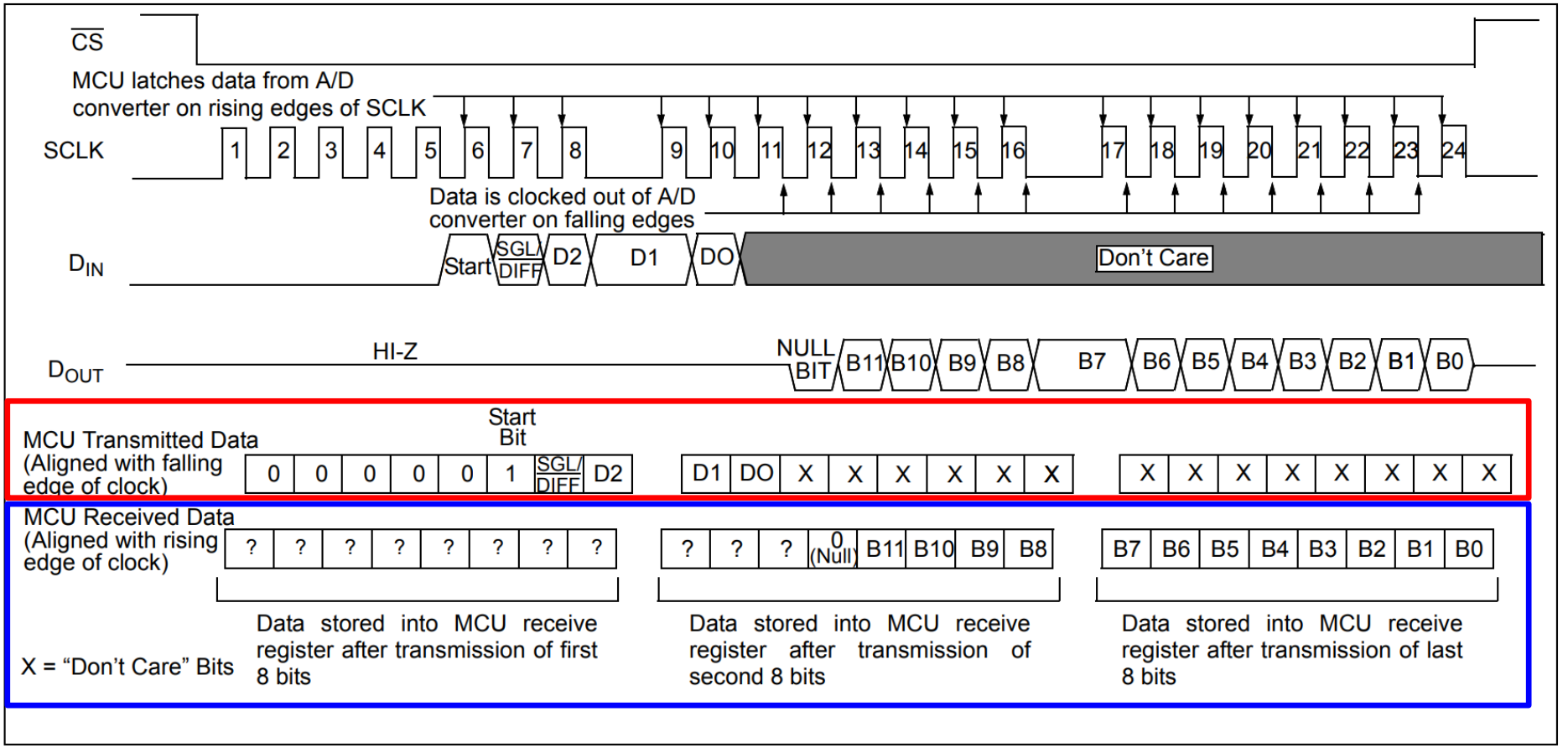


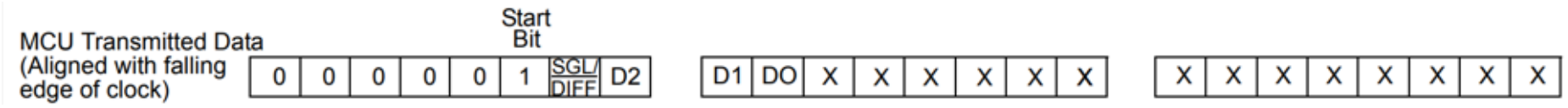

***FIGURE 6-1:*** *SPI Communication using 8-bit segments (Mode 0,0: SCLK idles low).*

**TABLE 5-2: CONFIGURATION BITS FOR THE MCP3208**

| Control Bit Selections | | | | Input Configuration | Channel Selection |
|---|---|---|---|---|---|
| Single /Diff | D2 | D1 | D0 | | |
| 1 | 0 | 0 | 0 | single-ended | CH0 |
| 1 | 0 | 0 | 1 | single-ended | CH1 |
| 1 | 0 | 1 | 0 | single-ended | CH2 |
| 1 | 0 | 1 | 1 | single-ended | CH3 |
| 1 | 1 | 0 | 0 | single-ended | CH4 |
| 1 | 1 | 0 | 1 | single-ended | CH5 |
| 1 | 1 | 1 | 0 | single-ended | CH6 |
| 1 | 1 | 1 | 1 | single-ended | CH7 |

# 9.1 การส่งคำสั่ง ไปยัง MCP3208

** จาก Datasheet พบว่าต้องส่งคำสั่ง ตามรูปแบบของ data sheet ดังนี้




โดยที่

D2 D1 D0 คือ Channel ที่ต้องการอ่านค่า

SGL/DIFF คือ Single end mode หรือ Differential mode

Single-ended Mode (SGL = 1)

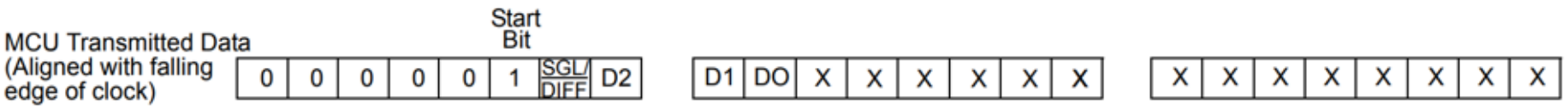
ADC จะวัดแรงดันของช่องที่เลือก เทียบกับ GND เช่น CH1

Differential Mode (DIFF = 0)

ADC จะวัด ความต่างศักย์ระหว่างสองช่อง เช่น CH1-CH0

# 9.1 การส่งคำสั่ง ไปยัง MCP3208

** จาก Datasheet พบว่าต้องส่งคำสั่ง ตามรูปแบบของ data sheet ดังนี้




ถ้าต้องการอ่านค่าจาก CH6 (110) แบบ Single end mode

จะต้องส่งคำสั่งไป 3 byte ดังนี้

0 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Start bit

D2 D1 D0

Single end

เขียนเป็นคำสั่งได้เป็น

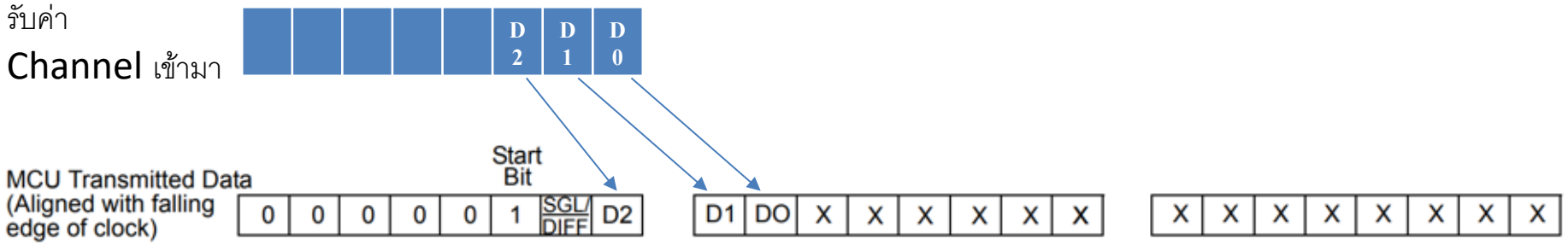
```
spi.xfer2([7,128,0])
```

# 9.1 การส่งคำสั่ง ไปยัง MCP3208

- สรุปเป็นคำสั่ง ของการส่งคำสั่งไปยัง mcp3208 ในการอ่านแต่ละ channel ดังนี้
  - Channel 0 `spi.xfer2([6,0,0])`
  - Channel 1 `spi.xfer2([6,64,0])`
  - Channel 2 `spi.xfer2([6,128,0])`
  - Channel 3 `spi.xfer2([6,192,0])`
  - Channel 4 `spi.xfer2([7,0,0])`
  - Channel 5 `spi.xfer2([7,64,0])`
  - Channel 6 `spi.xfer2([7,128,0])`
  - Channel 7 `spi.xfer2([7,192,0])`

# 9.1 การส่งคำสั่ง ไปยัง MCP3208

- ถ้าเขียนโค้ดเป็นแบบฟังก์ชั่น โดยรับค่า Channel เข้ามา ( มีค่า 0-7)
  - นำ D2 ไปเป็น bit 0 ของ byte แรก
  - นำ D1, D0 ไปเป็น bit 7 และ bit 6 ของ byte ที่สอง




จะเห็นว่า bit D2 ของ channel ต้องเลื่อนมาทางขวา 2 bit

และ bit D1, D0 ของ channel ต้องเลื่อนไปทางซ้าย 6 bit

(สนใจ bit D2 จึงต้อง AND ด้วย 100 , สนใจ bit D1,D0 จึงต้อง AND ด้วย 011)

**เขียนเป็น code ทั่วไปได้เป็น**

```
spi.xfer2([6 | (channel & 4) >> 2, (channel & 3) << 6, 0])
```

# 9.2 การอ่านค่า Data จาก MCP3208

** จาก Datasheet พบว่า MCP3208 ส่งข้อมูลกลับมา 3 Byte

สมมุติว่า MCP3208 อ่านค่าได้เท่ากับ 2748

แปลงเป็นเลข 12 bit ได้เท่ากับ

**1010 1011 1100**

MCP3208 จะส่งข้อมูลกลับมาเป็น 3 Byte

| Byte0 | Byte1 | Byte2 |
|---|---|---|
| -------- | -------- | -------- |
| xxxx xxxx | xxxx1010 | 10111100 |

หรือ **adc[0] adc[1] adc[2]**

โดยที่ adc[0] ไม่ได้ใช้

adc[1] เท่ากับ **x x x x D11 D10 D9 D8**

adc[2] เท่ากับ **D7 D6 D5 D4 D3 D2 D1 D0**

ดังนั้นข้อมูลที่อ่านได้ เขียนเป็นสมการได้เป็น

```
data = ((adc[1] & 15) * 256) + adc[2]
```

# 9.2 การอ่านค่า Data จาก MCP3208

**หรือ** สามารถเขียนอีกวิธีได้เป็น

```
data = ((adc[1] & 15) << 8) + adc[2]
```

อธิบายได้ว่า จาก

adc[1] เท่ากับ x x x x **D11 D10 D9 D8**

เลื่อนไปข้างหน้า 8 bit จึงกลายเป็น

adc[1] เท่ากับ x x x x **D11 D10 D9 D8 0 0 0 0 0 0 0 0**

จึงนำมารวมกับ **adc[2]** ได้

จาก

adc[1] เท่ากับ x x x x **D11 D10 D9 D8 0 0 0 0 0 0 0 0**
adc[2] เท่ากับ **D7 D6 D5 D4 D3 D2 D1 D0**

เมื่อรวมกัน จะกลายเป็น

**x x x x D11 D10 D9 D8 D7 D6 D5 D4 D3 D2 D1 D0**

# การทดลองที่ 3. Analog to Digital : MCP3208

**1.** พิมพ์คำสั่ง **raspi-config -> Interface option** to enable SPI

**2.** ติดตั้ง **Library**

```
$sudo apt-get install python3-pip
$sudo pip3 install spidev
```

**3.** สร้างไฟล์โดยใช้คำสั่ง **$nano test.py**

**4.** เขียนโปรแกรม

```
import time
import spidev
spi = spidev.SpiDev() # Open SPI bus
spi.open(0, 0)
spi.max_speed_hz = 500000
def ReadChannel(channel): # read channel (0-7) from MCP3208
    adc = spi.xfer2([6 | (channel & 4) >> 2, (channel & 3) << 6, 0])
    #ส่งข้อมูล3 byte ตามที่ Datasheet กำหนด แล้วรอรับค่าที่แปลงได้
    data = ((adc[1] & 15) << 8) + adc[2]
    return data
```

# การทดลองที่ 3. Analog to Digital : MCP3208

```
while True:
        for i in range(8):
                print('ADC[{}]: {:.2f}'.format(i, ReadChannel(i)))
        time.sleep(0.5)
```

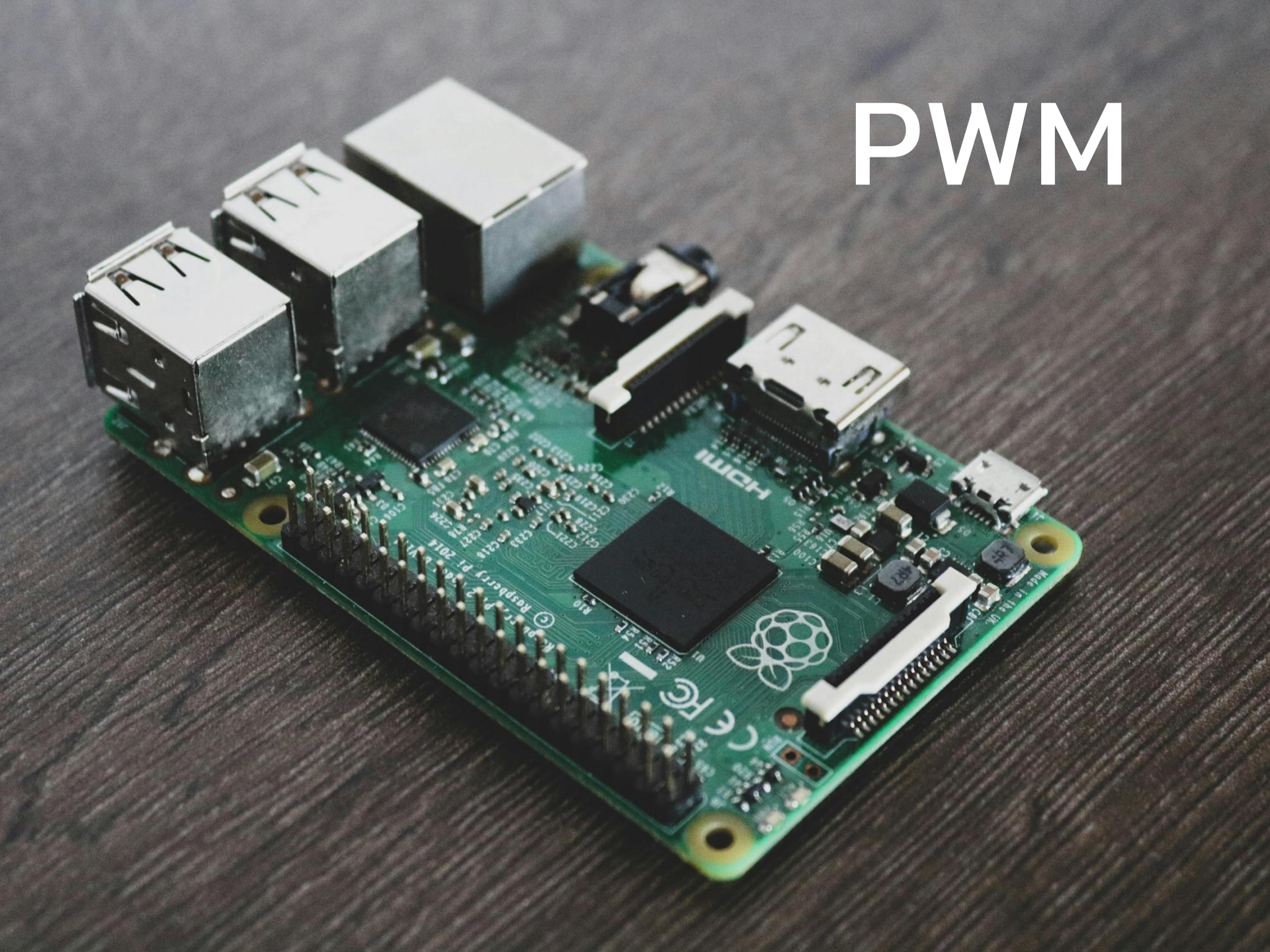
**หรือ**

```
while True:
    reading = ReadChannel(0)
    voltage = reading * 3.3 / 4096
    print("Reading=%d\t Voltage=%f" % (reading, voltage))
    time.sleep(1)
```

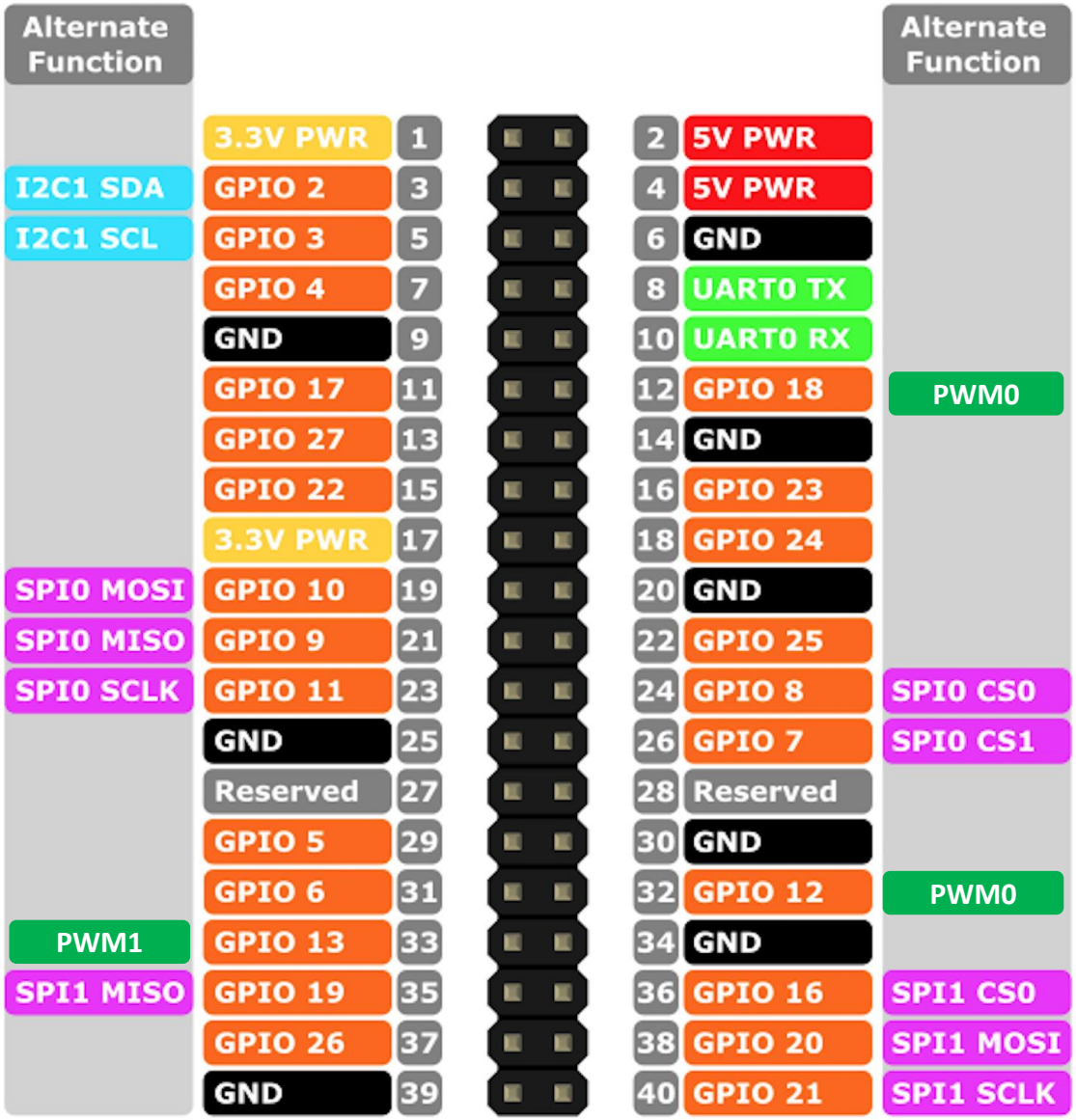
**5. Run** โปรแกรมโดยใช้คำสั่ง

```
$python3 test.py
```

จากนั้นลองหมุนตัวต้านทาน แล้วดูว่าอ่านค่าได้ 0-4095 และได้ค่าแรงดัน 0-3.3V


PWM

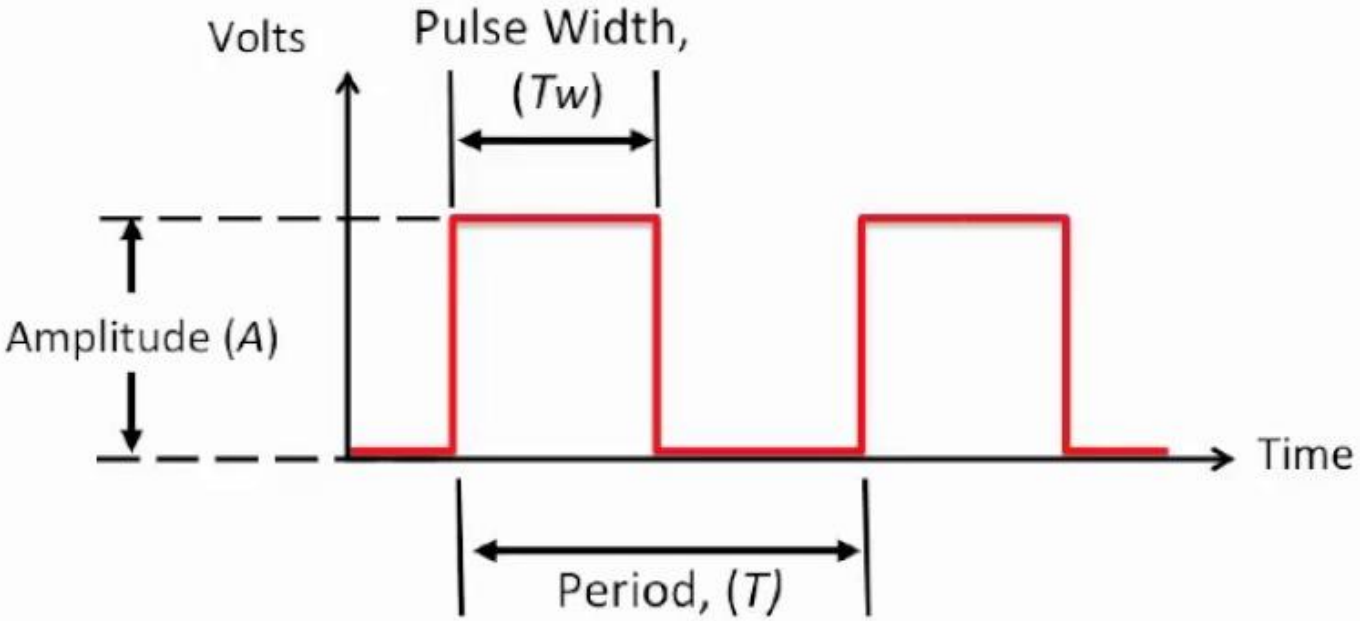
# GPIO Pin


Alternate Function
Alternate Function
3.3V PWR 1
2 5V PWR
I2C1 SDA
GPIO 2 3
4 5V PWR
I2C1 SCL
GPIO 3 5
6 GND
GPIO 4 7
8 UART0 TX
GND 9
10 UART0 RX
GPIO 17 11
12 GPIO 18
PWM0
GPIO 27 13
14 GND
GPIO 22 15
16 GPIO 23
3.3V PWR 17
18 GPIO 24
SPI0 MOSI
GPIO 10 19
20 GND
SPI0 MISO
GPIO 9 21
22 GPIO 25
SPI0 SCLK
GPIO 11 23
24 GPIO 8
SPI0 CS0
GND 25
26 GPIO 7
SPI0 CS1
Reserved 27
28 Reserved
GPIO 5 29
30 GND
GPIO 6 31
32 GPIO 12
PWM0
PWM1
GPIO 13 33
34 GND
SPI1 MISO
GPIO 19 35
36 GPIO 16
SPI1 CS0
GPIO 26 37
38 GPIO 20
SPI1 MOSI
GND 39
40 GPIO 21
SPI1 SCLK

# Waveform Characteristics

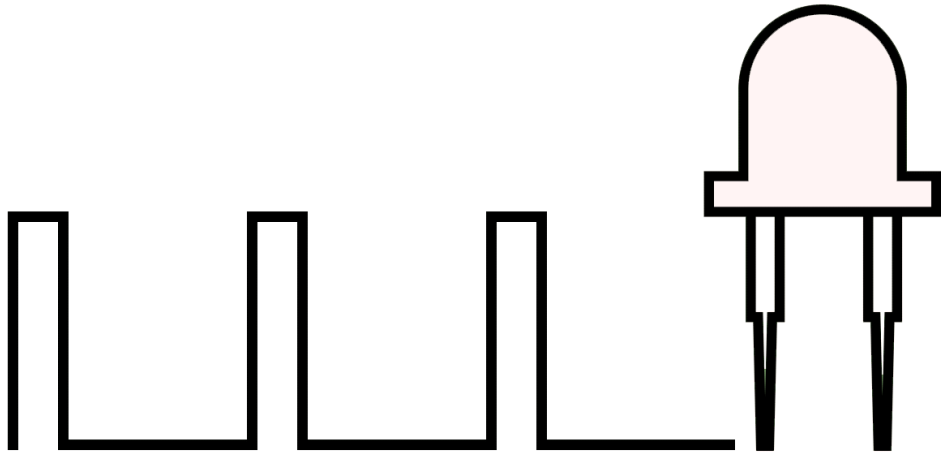
## Periodic Waveform **Duty Cycle**:

Duty Cycle = ratio of the Pulse Width to the Period



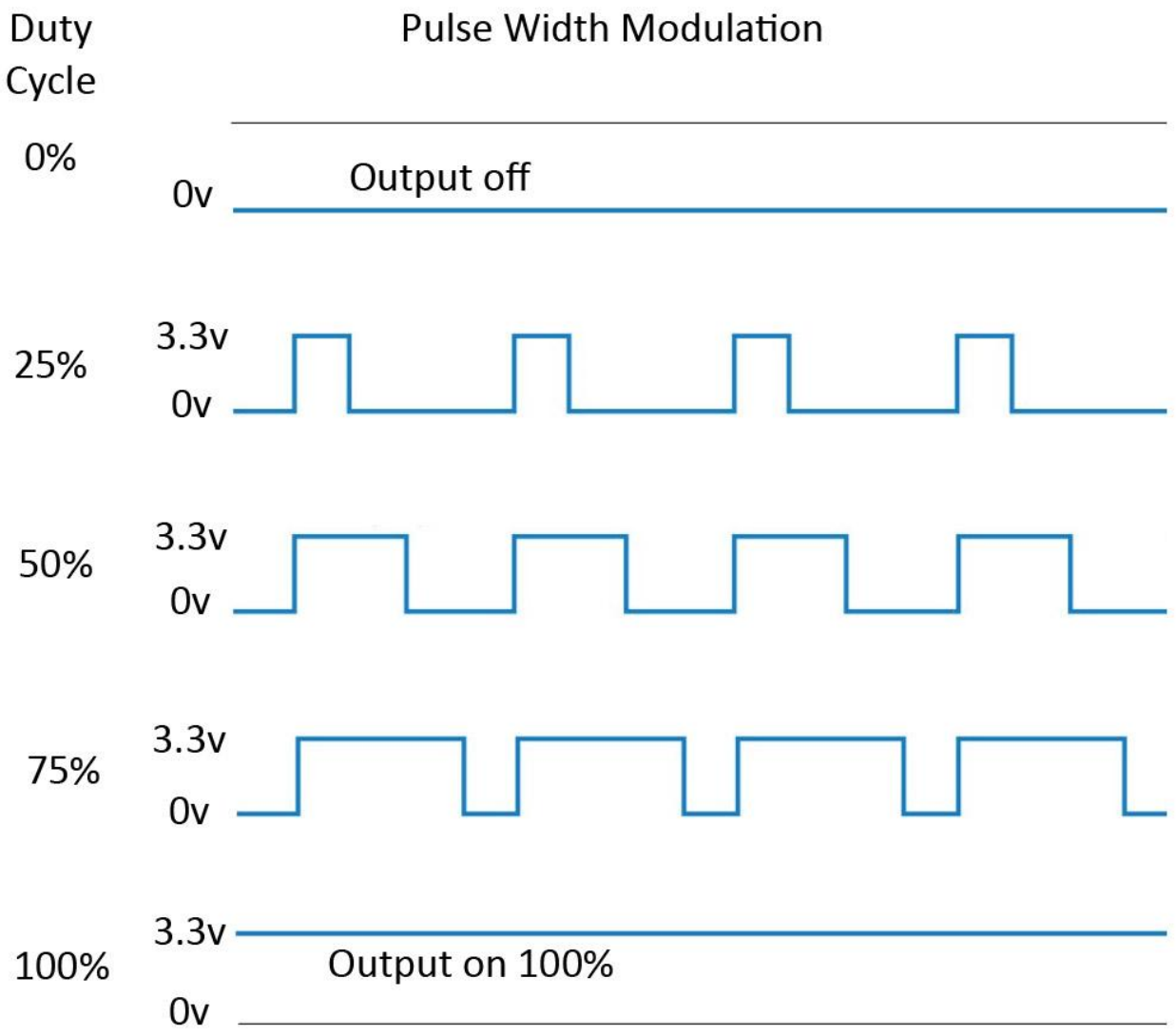
# PWM : Pulse Width Modulation

PWM คือ ค่า Duty Cycle มีค่าแปรผันตามแรงดัน input ที่ป้อนเข้ามา
โดยที่ค่า Duty cycle คือ ค่าความกว้าง Pulse ต่อ ค่าคาบเวลาของสัญญาณ



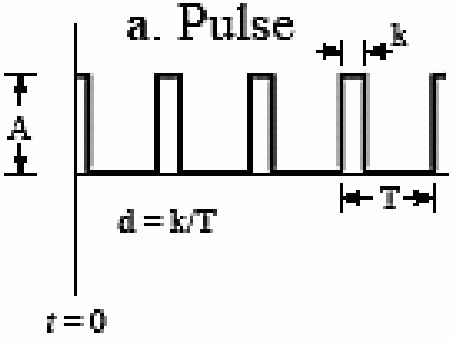
ถ้าค่า Duty cycle มีค่าน้อย ก็จะให้แรงดัน DC ออกมาน้อย
ถ้าค่า Duty cycle มีค่ามาก ก็จะให้แรงดัน DC ออกมามีค่าเยอะ
นิยม นำสัญญาณ PWM ไปใช้ควบคุม ความเร็ว motor เป็นต้น

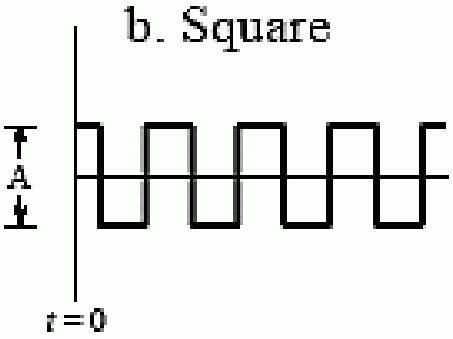
# Pulse Width Modulation



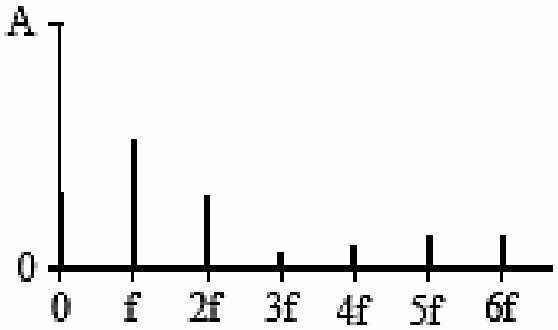
# Pulse Width Modulation

## Time Domain




## Frequency Domain

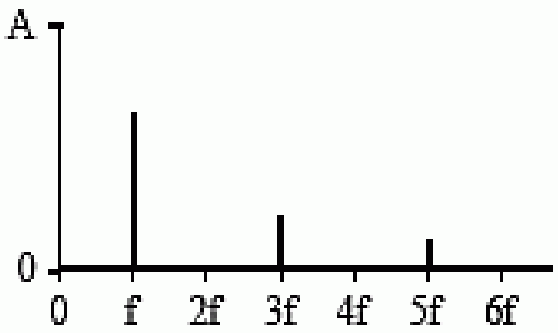



$$a_0 = A\,d$$

$$a_n = \frac{2A}{n\pi}\sin(n\pi d)$$

$$b_n = 0$$

($d$ = 0.27 in this example)




$$a_0 = 0$$

$$a_n = \frac{2A}{n\pi}\sin\left(\frac{n\pi}{2}\right)$$

$$b_n = 0$$

(all even harmonics are zero)

# Using PWM in GPIO

To create a PWM instance:

```
p = GPIO.PWM(channel, frequency)
```

To start PWM:

```
p.start(dc)  # where dc is the duty cycle (0.0 <= dc <= 100.0)
```

To change the frequency:

```
p.ChangeFrequency(freq)  # where freq is the new frequency in Hz
```

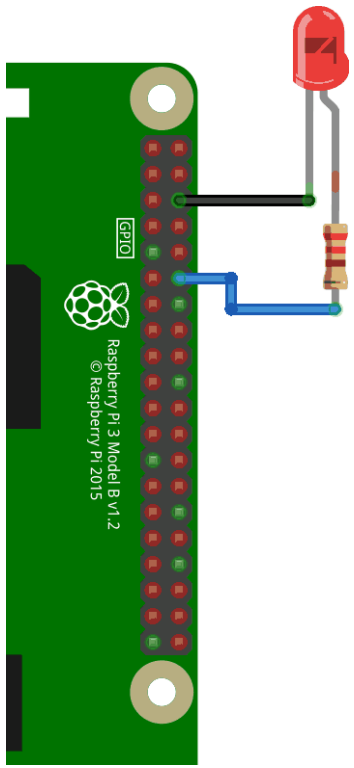
To change the duty cycle:

```
p.ChangeDutyCycle(dc)  # where 0.0 <= dc <= 100.0
```

To stop PWM:

```
p.stop()
```

# การทดลองที่ 4. Pulse Width Modulation




**1.**สร้างไฟล์โดยใช้คำสั่ง **$nano pwm.py**

**2.** เขียนโปรแกรม

```
import RPi.GPIO as GPIO
from time import sleep

GPIO.setwarnings(False)          #disable warnings
GPIO.setmode(GPIO.BOARD)         #set pin numbering
ledpin = 12                      #PWM pin connected to LED

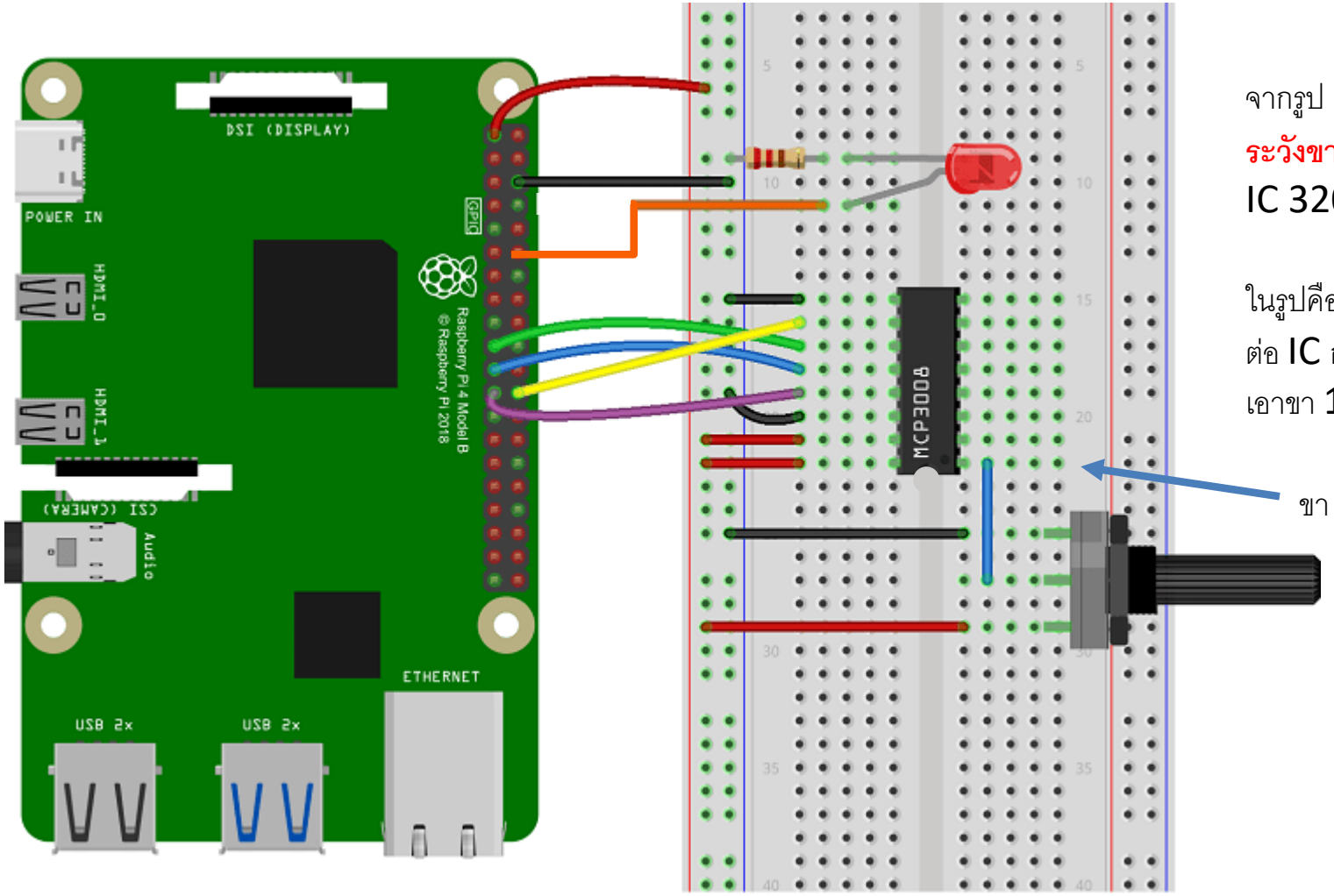
system GPIO.setup(ledpin,GPIO.OUT)
pi_pwm = GPIO.PWM(ledpin,1000)   #create PWM instance with frequency 1000
pi_pwm.start(0)                  #start PWM of 0% Duty Cycle

while True:
        for duty in range(0,101,1):
            pi_pwm.ChangeDutyCycle(duty)  #provide duty cycle in the
range 0-100
            sleep(0.05)
```

**3. Run** โปรแกรมโดยใช้คำสั่ง

**$python3 pwm.py**

# แบบฝึกหัดที่ 2. อ่านค่าจากความต้านทานปรับค่าได้ และปรับค่าความสว่าง LED




จากรูป

**ระวังขา**

IC 3208 ดีดี

ในรูปคือ

ต่อ IC กลับหัว

เอาขา 1 ไว้ล่าง

ขา 1


FINISH